

AART Slot Car Braking Module

Design Document

Target MCU: STM32G051K6U6 (QFN-32) | Firmware: Rev 11 | libopencm3

Revision 11 | 19 June 2026 | AART — Adrian & Richard's Technologies

PRIVATE & CONFIDENTIAL

With gratitude to Jeff Goldberg for his manifold contributions to this work.

PRIVATE & CONFIDENTIAL

1. Overview

This document describes the hardware design and firmware architecture for the AART slot car braking module. The module connects in-line between a conventional hand controller and the track, providing variable resistance braking, positive anti-brake for brushed motors, and a keep-alive voltage floor for brushless motors. All three functions are controlled by a three-position toggle switch and a single linear potentiometer, with fine tuning via UART commands.

1.1 Purpose

The AART brake module provides three capabilities, controlled by a three-position toggle switch, a linear potentiometer, and UART commands:

- Braking — for brushed and brushless motors. Q2 shunts a variable resistance across the motor terminals. The pot governs brake force from a settable minimum (selectable 25/15/5/4/3/2/1 Ω via SET BRAKE_OHMS, full CCW) to dead short (full CW). Fine electronic control replaces the fixed resistor of a conventional brake.
- Anti-brake — for brushed motors (Mode B). While braking, Q1 injects a small forward current from the WHITE rail that opposes the brake, countering the back-EMF that makes the car snap on corner entry. The pot simultaneously governs both brake resistance and anti-brake level through a lookup table: lighter braking gives more anti-brake, harder braking less.
- Keep-alive floor — for brushless motors (Mode A with floor set). The eCom is powered from the braid. Conventional braking collapses the braid voltage to near zero, resetting the eCom. Q1 clamps the braid above a settable floor, preventing brownout. The floor is set by rotating the pot in Mode C then toggling to Mode A.

Parameters requiring fine or infrequent adjustment — ramp-in time, trail brake release, brake entry/exit thresholds, and Latvian brake — are set via UART and saved to flash.

The module is fully compatible with any conventional hand controller and requires no modification to the controller or the car.

1.2 System position

The module connects via 4mm banana plugs to the three standard track sockets and exports matching sockets for the hand controller to plug into. It is fully transparent during normal running and takes over braking when the controller shorts the track.

Banana colour	Signal	Description
WHITE	+12–16V always-live rail	From track PSU. Can deliver 40–90A. Powers the module and Q1 injection.
BLACK	Track positive (+ve to	Controlled by hand controller. Monitored by module to

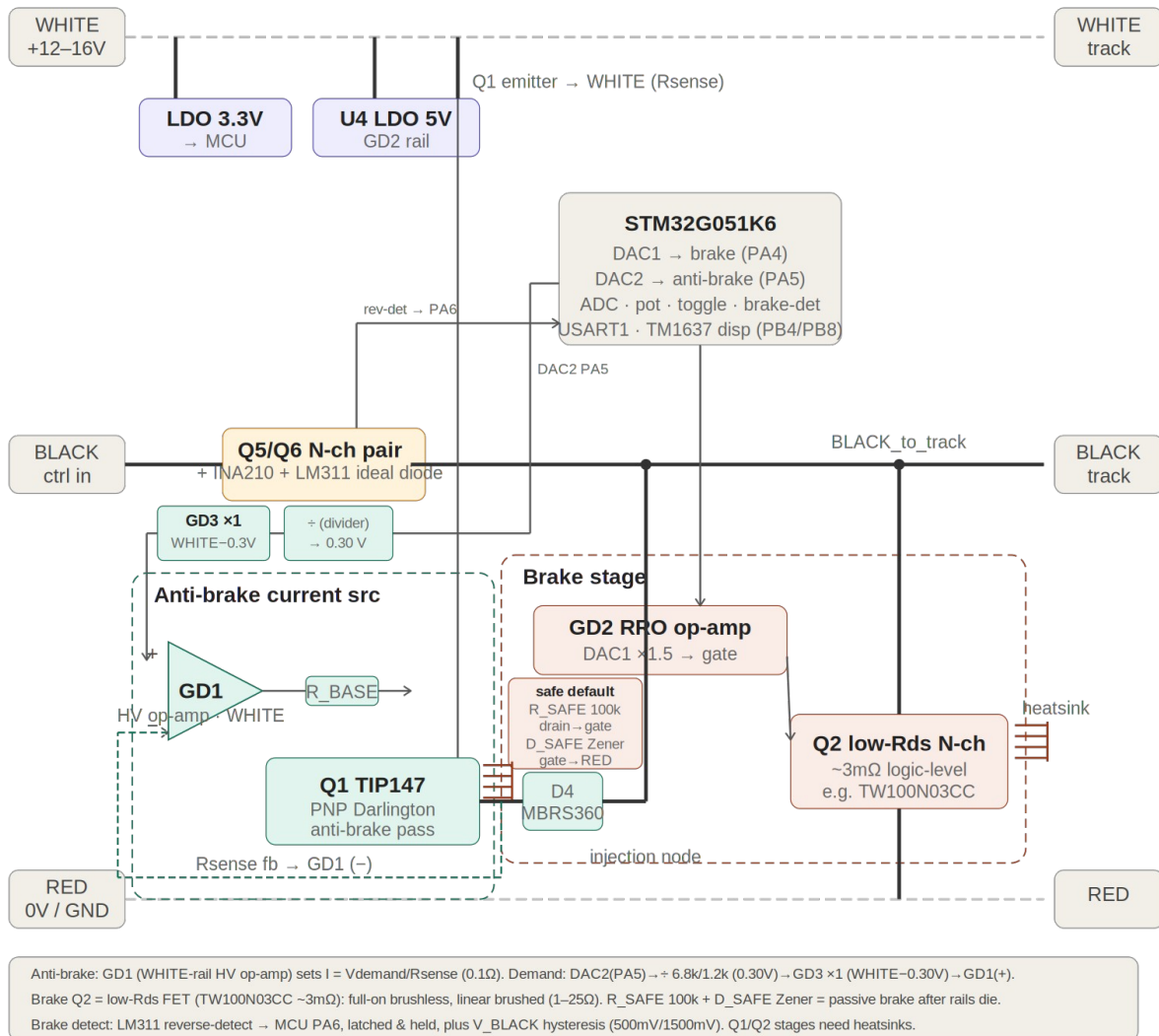
Banana colour	Signal	Description
	car)	detect brake condition.
RED	Common negative	Left braid return. Module and track ground reference.

The module presents identical WHITE/BLACK/RED sockets on its output side for the hand controller to plug into. During normal running, BLACK passes through unmodified. When the hand controller shorts BLACK→RED (brake condition), the module detects this by voltage sense on the BLACK terminal and applies its own controlled braking.

The WHITE rail is always live at 12–16VDC regardless of throttle position. The hand controller varies how much of this rail reaches the BLACK output. The module draws its MCU supply from WHITE via the LDO. WHITE is also the source for Q1 (anti-brake injection and keep-alive floor).

1.3 Circuit topology

The diagram below shows the principal signal paths through the module. All three rails (WHITE, BLACK, RED) run left to right. The hand controller connects on the left; the track and motor are on the right.



[Figure 1: Circuit topology diagram — see AART_Brake_Topology.svg delivered alongside this document]

Key points from the diagram:

- The reverse-blocking switch (two N-channel + INA210/LM311 ideal diode, section 7A.2) sits in the BLACK controller input path — not the WHITE rail. During normal running it is fully transparent (near-zero voltage drop). During brake takeover it opens, isolating the controller's short from the track.

Note: The reverse-blocking element is a two N-channel common-source bidirectional switch controlled as an ideal diode by an INA210 current-sense amp and an LM311 comparator (both on +5V), not an anode-powered ideal-diode IC — see section 7A.2. The LM311 turns the pair off on reverse current in ~2μs (sim), with a ~0.16A trip threshold and ~50mV hysteresis. This drops the automotive I^2t / 5μs short-

circuit / IMON protection of the earlier TPS1212-Q1 (open item). Insertion drop ~87mV at 40A including the sense resistor.

- Q1 (TIP147 PNP Darlington) is the pass device of the anti-brake / keep-alive constant-current source: the WHITE-rail op-amp GD1 regulates the current through Q1 and R_{sense} (0.1Ω) to $I = V_{demand}/R_{sense}$, up to ~3A. In Mode A it holds the saved keep-alive floor whenever V_{BLACK} drops below it; in Mode B it injects anti-brake current after brake exit; in Mode C it previews the floor live while the driver tunes it. The demand is a clean DC voltage from DAC2 (DAC_OUT2 on PA5), divided down to set the target (section 7A.7).
- Q2 (low- R_{ds} logic-level N-channel, low-side) connects BLACK_to_track to RED/GND. For brushed motors it is held in its linear region at a selectable brake resistance (25/15/5/4/3/2/1 Ω via SET BRAKE_OHMS, down to a near dead short); for brushless motors it is driven full-on, shorting the on-car ESC bus so the motor brakes through the Remora's body-diode rectifier. Its gate is driven by GD2, an op-amp buffer with gain ~1.5, from DAC1 (sections 7A.10–7A.12).
- The track-facing devices never hard-switch. For the brushed brake the DAC sets Q2's gate to a DC voltage, so it presents a continuously variable drain-source resistance — no 20kHz switching, no radiated EMI, no interference with brushless eCom electronics; for the brushless brake Q2 is simply held fully on. The anti-brake stage is a constant-current source whose demand is a clean DC voltage from DAC2, so nothing on the track node switches at all.
- The MCU senses WHITE dV/dt (for Latvian brake detection), $V_{BLACK_to_track}$ (for brake entry/exit hysteresis), and the pot and mode switches. It generates the DAC1 brake voltage and the 50kHz anti-brake PWM demand accordingly.
- The R10/R11 voltage divider sensing BLACK_to_track is on the motor side of the reverse-blocking pass switch — this is essential so the MCU can detect the voltage collapse that occurs when that switch opens and the module takes over braking.

1.4 Circuit-by-circuit description (theory of operation)

A guided tour of each functional circuit in the module, in signal-flow order — what it does, how it works, and its key parts. The detailed analysis of each lives in the referenced sections; the topology diagram (§1.3) shows how they connect.

Power input and rails — Three rails run through the module: WHITE (the 12–16 V track PSU, always live), BLACK (the controller-to-track drive/signal, swinging 0→rail) and RED (0 V/GND). An LDO derives 3.3 V for the MCU; a second LDO (U4) derives the +5 V GD2 rail. A 100 μ F hold-up capacitor keeps the MCU and brake stage alive through a PSU power-off, so braking still works when track power is cut (§7A.6).

Reverse-blocking pass switch — Q5/Q6, a back-to-back N-channel pair in the BLACK controller-input path, controlled as an ideal diode by an INA210 + LM311 (both on +5V). It passes forward controller current at a low insertion drop and blocks reverse current in ~2 μ s; the LM311 output also drives the MCU brake-detect on PA6. It sits in the BLACK path, not the WHITE rail (§7A.2).

MCU and power supervisor — An STM32G051K6 (QFN-32) at 64 MHz runs the control loop. It senses WHITE dV/dt (Latvian-brake trigger), $V_{BLACK_to_track}$ (brake entry/exit hysteresis), the control pot and the mode toggles; and it drives DAC1 (brake), DAC2 (anti-brake demand), the TM1637 display (PB4/PB8), the status LED, USART1 (telemetry/flashing) and SWD. Its BOR/POR supervisor plus the hold-up cap let braking survive a brownout (§7A.5, §7A.6).

Brake detection — The R10/R11 divider senses V_BLACK_to_track on the motor side of the pass switch, and firmware applies hysteresis to decide brake entry and exit — when the controller is released and the drive collapses, the module enters braking (§2.1).

Brake stage (Q2 + GD2) — Q2 is a low-R_{ds} logic-level N-channel FET, low-side from BLACK_to_track to RED. Its gate is driven by GD2, an LM7321 rail-to-rail-output op-amp on +5 V with a gain of ~1.5 that lifts the 3.3 V DAC to a ~5 V gate. For brushless motors Q2 is driven full-on — shorting the on-car ESC bus, which brakes through the Remora's body-diode rectifier; for brushed motors it is held in its linear region at a selectable resistance (the 1–25 Ω tables). A catch diode (D2) and heatsink complete the stage (§7A.10–7A.12, §11.2).

Anti-brake / keep-alive current source (GD1 + Q1) — GD1, an LM7321 rail-to-rail-input op-amp on the WHITE rail, closes a constant-current loop around Q1 (a TIP147 PNP Darlington pass device) and R_{sense} (0.1 Ω). A clean DC demand from DAC2 (DAC_OUT2 on PA5) sets the target, and GD1 regulates $I = V_{\text{demand}}/R_{\text{sense}}$ up to ~3 A, injecting a controlled forward current into BLACK_to_track — positive anti-brake in Mode B, and a keep-alive floor for brushless motors that keeps the ESC responsive (held in Mode A, tuned live in Mode C). D4 (MBRS360) blocks reverse (§7A.7).

Digital display — A TM1637 4-digit LED module on J14, driven directly by the MCU over a 2-wire serial link (CLK = PB4, DIO = PB8), shows the WHITE rail voltage in Modes A/B and the live keep-alive floor in Mode C. The MCU sends exact digits, so there is no analog proxy, RC filter, or scaling.

USB-C / UART bridge — A CH340N USB-to-UART bridge on J11 carries telemetry and runtime SET commands; field firmware updates run through the STM32 ROM UART bootloader using the BOOT0/NRST buttons (§7A.9, §14).

2. Operating Modes

The module has three operating modes selected by a 3-position toggle switch. Within each mode the brake is triggered by voltage sense on the BLACK terminal rather than a mechanical wiper strip contact, making the module compatible with any hand controller.

2.1 Brake detection — current direction (I_DIR) with V_BLACK hysteresis

Two signals drive brake detection. The primary is the reverse-detect flag on PA6 (the LM311 ideal-diode output): reverse current through the pass switch means the controller is braking, latched and held across brief reverse-detect dropouts. The BLACK terminal voltage (PA1) is also monitored, with a hysteresis band, to catch coast/lift-off and to exit when the controller drives forward again:

- I_DIR asserted (reverse current) → enter brake state and hold — controller is braking
- V_BLACK < 500mV → enter brake state (controller has shorted BLACK→RED)
- V_BLACK > 1500mV → exit brake state (controller delivering power)
- 500mV ≤ V_BLACK ≤ 1500mV → hold current state

This approach works with any hand controller regardless of whether it has an accessible wiper strip contact, and correctly handles partial braking (controllers that do not short fully to zero).

2.2 Mode A — Brake only (brushed and brushless)

V_BLACK	Pot position	Q1 (KA floor)	Q2 (brake)	Behaviour
> 1500mV	Any	OFF (KA floor injects only when V_BLACK drops below it)	OFF	Pass-through — module transparent. Q1 holds KA floor if configured.
< 500mV	Full CCW	OFF (during braking)	Configurable soft min (SET BRAKE_OHMS)	Softest brake at the selected resistance (25/15/5/4/3/2/1Ω).
< 500mV	CCW → CW	OFF (during braking)	Rising toward DAC_BRAKE_HARD	Brake firms progressively as pot turns CW.
< 500mV	Full CW	OFF (during braking)	DAC_BRAKE_HARD	Dead short — maximum braking.
< 500mV → released	Any	saved_ka_dac applied when V_BLACK < floor	OFF	Q1 maintains keep-alive floor (brushless). Default DAC_KA_OFF = pure braking.

2.3 Mode B — Brake and anti-brake (brushed motors)

State (detection)	Pot	Q1 (anti-brake)	Q2 (brake)	Net behaviour
Driving forward (I_DIR clear, V_BLACK high)	Any	OFF	OFF	Pass-through. No anti-brake injected while the driver is on throttle.
Braking (I_DIR reverse current, or V_BLACK collapse)	Full CCW	ka_dac_table[idx]	brake_tables[sel][idx] (soft)	Q1 and Q2 act together: anti-brake trims the brake most → softest net brake.
Braking	CCW → CW	ka_dac_table[idx]	brake_tables[sel][idx] (rising)	Net brake firms progressively as the pot index climbs.
Braking	Full CW	ka_dac_table[idx]	brake_tables[sel][idx] (dead)	Q2 dominates → hardest net brake.

PRIVATE & CONFIDENTIAL

State (detection)	Pot	Q1 (anti-brake)	Q2 (brake)	Net behaviour
			short)	

2.4 Mode C — Keep-alive tuning (brushless)

Mode C is a setup mode for brushless motors only. The pot dials the keep-alive floor voltage live (ka_dac_table[pot_idx]) which is applied to Q1 so the driver can verify the eCom powers up correctly. When the toggle leaves Mode C, the current pot value is saved as saved_ka_dac after a 500ms debounce. This saved value is then used by Mode A as the Q1 KA floor. Return to Mode A or B for racing.

V_BLACK	Pot position	Q1 (KA preview)	Q2 (brake)	Behaviour
Mode C active, V_BLACK high	Any	OFF	OFF (suppressed)	Q1 previewing floor, pot CCW = 0V floor
Mode C active, V_BLACK low	Any	OFF	Linear per pot (brake)	Braking + KA floor preview simultaneously
V_BLACK > 1500mV	Any	ka_dac_table[pot_idx] live	OFF	Q1 live, eCom can verify floor
V_BLACK < 500mV	Any	OFF (Q1 off during braking)	Linear per pot	Braking, pot = configurable soft → DAC_BRAKE_HARD
Toggle away from C	Any	OFF	Normal	500ms debounce then save + LED 3×
Back in Mode A	Any	saved_ka_dac applied as floor	Linear per pot	Racing with KA floor active

During a hard brake (Q2 dead short), V_BLACK collapses toward 0V. Q1 is commanded off during braking in Mode C (per the table above), so it is not maintaining the floor here; the low impedance of Q2's dead short would dominate in any case. D4 is essential in mode C — it blocks reverse current through Q1 when Q2 holds the track near 0V, protecting Q1's body diode from the repetitive peak current.

3. Hardware Design

3.1 Component list

Ref	Component	Value / Part	Notes
U1	MCU	STM32G051K6 U6	QFN-32, 3.3V, 64MHz
U2	LDO regulator	AMS1117-3.3 (or MCP1700 — see 7A.6)	WHITE rail → 3.3V, SOT-223
GD1 (HV op- amp power ed from the WHIT E rail) is the curren t- sourc e loop drivin g Q1 TIP14 7; Rsens e 0.1oh m; D4 MBR S360 revers e block	Anti-brake current source	LM7321 RRIO HV op-amp (SOT-23); alt OPA192	Keep-alive / anti-brake FET channel (Q1), SOT-23-5
Rsens e	Current sense	0.1ohm (1-2W)	0.3V at 3A -> op-amp sets $I = V_{\text{demand}} / R_{\text{sense}}$
GD3	Demand translator	LM7321 RRIO HV op-amp (SOT-23)	WHITE-referenced unity difference translator: $AB_DEMAND = WHITE - 0.30V$ ($R_{t1} - R_{t4} = 10k$ 0.1% matched; combinable with GD1 as one LM7322 dual)
R_KA 1/2	Demand divider	6.8k / 1.2k	Ground-referenced demand divider → 0.30V into GD3
GD2	Brake gate buffer	LM7321 RRO op-amp, gain ~1.5 (SOT-23); alt TLV9061	Brake FET channel (Q2), SOT-23-5

PRIVATE & CONFIDENTIAL

Ref	Component	Value / Part	Notes
Q1	Anti-brake / keep-alive FET	TIP147 PNP Darlington	Anti-brake current-source pass, TO-220 + heatsink (or N-MOSFET alt)
Q2	Brake FET	TW100N03CC ~3mΩ logic-level N-ch	30V, ~3mΩ, D2PAK SMD
Q5, Q6	Reverse-block N-ch pair	STL059N4S8A G (or SiJK140E)	Common-source bidirectional switch, BLACK path. 40V, ~0.59mΩ each.
U5	Smart high-side switch controller	INA210 + LM311	INA210 (gain 200, +5V) across Rsns_sw; LM311 (~50mV hysteresis) trips on reverse (~0.16A, ~2μs) → gate driver + PA6 brake-detect. No I ² t/SCP/IMON. Gate-drive charge pump from WHITE (open item). SC70-6 / SOT23-5.
Rsns_sw	Sense resistor (Rsns_sw)	~0.5-0.75mohm	Ideal-diode shunt; INA210 senses across it. ~40mV at 40A (~1mΩ). Sets the reverse threshold.
U7	Glue logic	1 transistor/gate	LM311 reverse-detect → gate driver off on reverse; output also feeds PA6 brake-detect.
R_TH R1/ R_TH R2, R_HY S, R_CM P	LM311 threshold + hysteresis	caps + resistors	Set the reverse trip (~0.16A) via the R_THR divider and ~50mV via R_HYS; R_CMP pulls the LM311 output up to +3V3.
D1	TVS diode	P6KE18CA (bidirectional)	Rail transient clamp, 18V, SMB. CA suffix essential — bidirectional variant clamps both polarity transients from inductive switching.
D2	Schottky	SS310	Q2 catch diode, 3A/100V SMB. 100V reverse voltage gives margin over SS34's 40V on a 16V rail with inductive spikes.
D3	Schottky	SS310	Q1 catch diode, 3A/100V SMB. Same upgrade as D2.
D4	Schottky	MBRS360	Mode C series output diode — ESSENTIAL
D5	LED red	0805	Status indicator PB6
F1	Polyfuse	500mA–1A, 1812	WHITE rail supply protection for MCU/LDO only. Must NOT be in the BLACK (track) path — a polyfuse cannot handle 40A motor startup and will trip from thermal lag. The BLACK path carries full motor current; PSU overcurrent protection covers that rail.
R1	10kΩ	0402	NRST pull-up
R2	10kΩ	0402	GD2 input pull-up (safe default brake)
R3	10Ω	0402	Q2 gate resistor

PRIVATE & CONFIDENTIAL

Ref	Component	Value / Part	Notes
R_BA SE	1kΩ	0402	Q1 base resistor (TIP147)
R5	10kΩ	0402	Pot top-end to 3.3V
R6	10kΩ	0402	PB3 reserved (no component fitted)
R7	1kΩ	0402	PB3 reserved — 1kΩ footprint, DNF
R8	18kΩ	0402	WHITE sense top divider (PA0)
R9	4k7Ω	0402	WHITE sense bottom divider (PA0)
R10	22kΩ	0402	BLACK sense top divider (PA1)
PA6	I_DIR (brake detect)	GPIO input	LM311 reverse-detect flag (ideal diode); current-direction brake detection. CONFIRM level ≤3.3V (pull-up to +3V3).
R11	4k7Ω	0402	BLACK sense bottom divider (PA1)
R12	330Ω	0402	Status LED current limit
C1– C4	Decoupling	0805/0402	100nF + 10μF per LDO rail
C5– C7	Decoupling	0402	100nF VDD, 100nF VDDA, 4.7μF VDDA
J1–J3	4mm banana sockets	Track side	WHITE / BLACK / RED to track
J4–J6	4mm banana sockets	Controller side	WHITE / BLACK / RED from hand controller
J7	3-pin 2.54mm	Pot connector	3.3V / wiper / GND
J8	3-pin 2.54mm	Mode toggle	Bit0 / Bit1 / GND
J9	2-pin 2.54mm	(reserved, was capture button)	Momentary pushbutton

3.2 Pin assignments — STM32G051K6U6 QFN-32

Pin	Signal	Type	Function
PA0	WHITE sense	ADC_IN0 analog	WHITE rail ÷ R8/R9 divider. $V_{PA0} = V_{WHITE} \times 0.2386$. Safe to 16V.
PA1	BLACK sense	ADC_IN1 analog	BLACK_to_track ÷ R10/R11 divider. Track side of the pass switch.
PA3	Pot wiper	ADC_IN3 analog	10k linear pot, 0–3.3V. No centre detent.
PA4	Q2 brake gate	DAC_CH1	0–3.3V → GD2 op-amp → Q2. DAC_BRAKE_OFF... DAC_BRAKE_HARD.
PA5	Q1 KA	DAC_OUT2	DAC2 (DAC_OUT2 on PA5) → ground-referenced

PRIVATE & CONFIDENTIAL

Pin	Signal	Type	Function
	demand (DAC2)	(analog)	divider (6.8k/1.2k → 0.30V) → GD3 difference translator → GD1 current-source op-amp → Q1 TIP147. Sets anti-brake current (0=off ... 3A). (DAC2 supplies clean DC; no PWM/RC filter.)
PA9	USART1_TX	AF1	UART TX to J11 / CH340N. 115200 8N1.
PA10	USART1_RX	AF1	UART RX from J11 / CH340N.
PB0	Toggle bit 0	GPIO input, pull-up	Active LOW. Decoded with PB1: see toggle table.
PB1	Toggle bit 1	GPIO input, pull-up	Active LOW. Decoded with PB0: see toggle table.
PB3	(reserved)	GPIO	Previously capture button. Unused in Rev 11.
PB4	TM1637 CLK	GPIO OD out	Display clock → J14 (TM1637)
PB6	Status LED	GPIO output	Blinks 3× on keep-alive save to flash.
PB8	TM1637 DIO	—	Display data → J14 (TM1637). PB8 freed by the BOOT0 = PA14 correction.

3.3 Voltage dividers

Both sense inputs use the same bottom resistor (4k7Ω) for component commonality. The top resistor differs to accommodate the different maximum voltages:

Signal	R_top	R_bot	Ratio	V_pin at 16V rail	Safe?
WHITE (PA0)	15kΩ	4k7Ω	0.2386	3.82V — marginally over	Over 3.3V — raise top to 18kΩ (revised row below)
WHITE (PA0) revised	18kΩ	4k7Ω	0.2073	3.32V	✓ safe to 3.3V rail
BLACK (PA1)	22kΩ	4k7Ω	0.1760	2.82V	✓ safe to 3.3V rail

If the track PSU can deliver up to 16V, use R8=18kΩ (not 15kΩ) for the WHITE sense divider. Update WHITE_DIV_NUM/DEN in brake_module.h to 47/234 accordingly. The original 15k value was sized for 12.4V supplies; at 16V it produces 3.82V on PA0 which exceeds the 3.3V ADC reference.

3.4 Safe default behaviour

On power loss or MCU fault, the brake FET (Q2) must default to ON (hard brake). This is achieved by a pull-up resistor on the Q2 op-amp buffer input. When the MCU is unpowered, the buffer output rises to the braking region and Q2 conducts.

- When the rails die, R_SAFE (100kΩ from Q2 drain to gate) couples the coasting motor's back-EMF on BLACK_to_track to the gate, turning Q2 on in proportion to car speed,

PRIVATE & CONFIDENTIAL

while D_SAFE (Zener gate-to-source) clamps the gate. The result is a passive, self-limiting back-EMF brake — firm at speed, fading as the car slows, and off when the car is stationary — not a fixed linear hold.

- brake_force_safe() writes directly to the DAC registers and is called from hard_fault_handler and nmi_handler.
- The hold-up capacitor (C_HU 100μF on 3.3V rail) maintains MCU power for ~300ms after PSU cut, allowing the firmware to maintain the full programmed brake characteristic through power-off.

3.5 Power and thermal analysis

The following analysis confirms component ratings against the worst-case electrical demands of the application. Track PSU: 12.4V nominal, 16V maximum. Motor startup: 40A for a few milliseconds. Braking back-EMF: 20A+ momentarily. PSU cannot absorb reverse current.

Component	Worst-case demand	Rating	Verdict	Action taken
Q2 TW100N03 CC (brake FET)	Linear operation. Worst-case mid-range ($R_{ds} \approx$ motor R): ~12W peak, intermittent (~10–20% brake duty). HEATSINK REQUIRED — see 7A.8. Low- R_{ds} logic-level part (e.g. TW100N03CC, ~3mΩ): full-on for brushless, linear for the brushed tables — see §7A.11–7A.12 for the effectiveness/thermal and linear-SOA analysis (which supersede the ~12W IRFP250N figure above).	≥30V, ~3mΩ logic-level, SMD power package + heatsink	✓ Comfortable	No change. Motor startup current (40A) does not flow through Q2 — Q2 is OFF during throttle. Copper pour under D2PAK pad recommended.
Q1 TIP147 (anti-brake current source)	Anti-brake current source (3A): pass device drops ~full rail at stall ≈ 47W worst case (transient, not continuous). HEATSINK REQUIRED — see 7A.8. TIP147 PNP Darlington (or N-	18A, 55V, 175°C T_j hFE ≈ 1000 (PNP Darlington). At stall Q1 is linear with VCE ≈ full rail — the source of the ~47W;	✓ OK (with D4)	D4 mandatory in mode C. Without D4, body diode of Q1 carries 46A repetitively at 20kHz — not rated for this. D4 blocks this current path. The 47W is only the brief brake-onset transient (motor still spinning, high back-EMF); a sustained stall must be prevented in firmware by collapsing the KA demand when V_BLACK fails to rise — validate against the TIP147

PRIVATE & CONFIDENTIAL

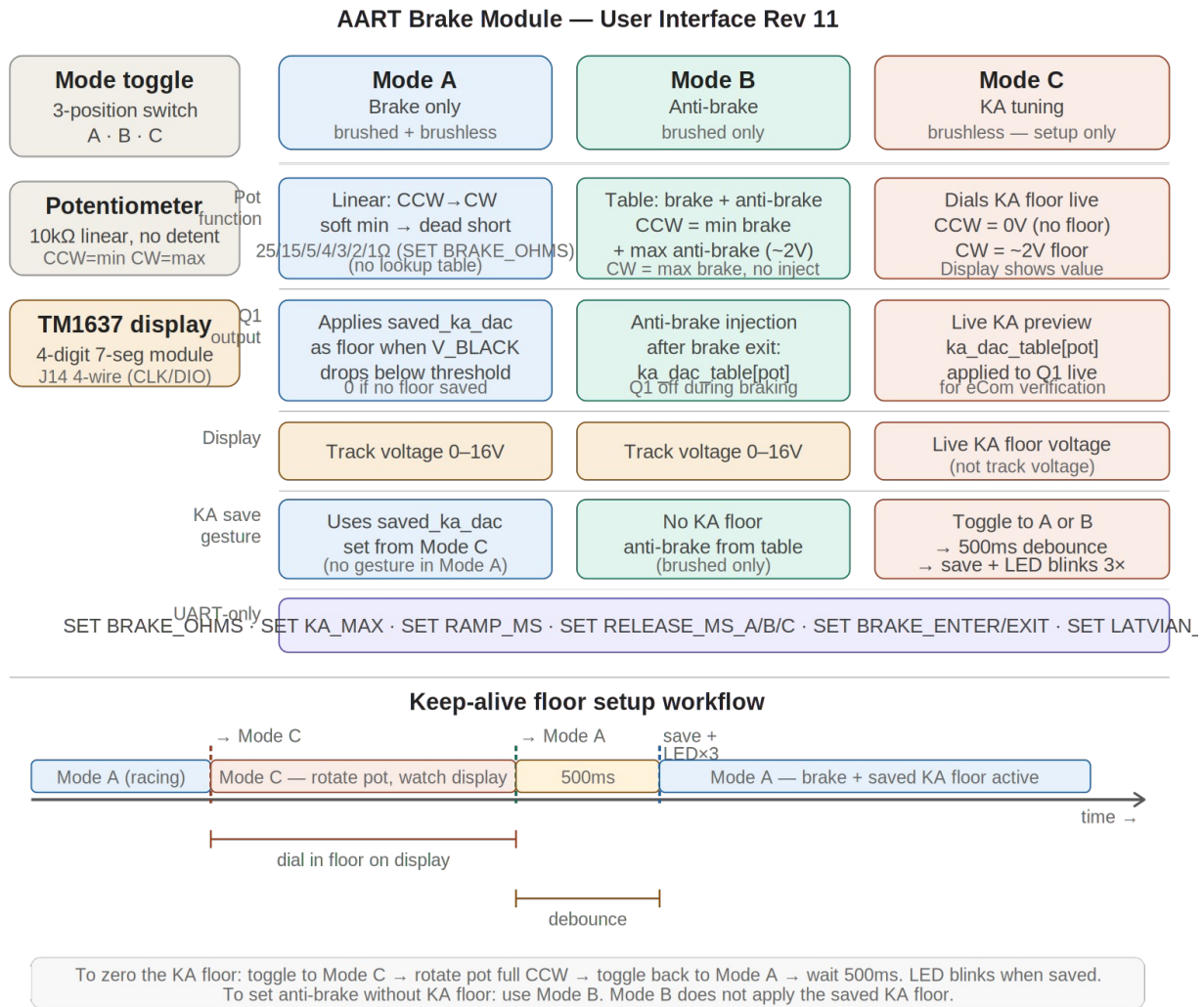
Component	Worst-case demand	Rating	Verdict	Action taken
	MOSFET); D4 blocks reverse current.	$V_{CE(sat)} \approx 2.5V$ applies only when saturated (low V_{CE}), not when sourcing into Q2's short		DC-SOA on the bench.
D1 TVS (P6KE18CA)	Motor back-EMF adds to rail: 16V + 12V peak = 28V transient Energy: $\ll 600W \times 1ms$ pulse capacity	600W peak (10/1000 μs), $V_c = 25.2V @ 1A$	✓ OK	Changed to bidirectional CA variant. Unidirectional P6KE18A only clamps one polarity; inductive switching generates both.
D2/D3 SS310 (catch diodes)	Q2 turn-off spike: $L \times di/dt = 100\mu H \times 20A/1\mu s = 2000V$ unclamped D2 clamps at $V_f + V_{dc} \approx 16.4V$, energy = 20mJ	3A avg, 100V, same SMB footprint as SS34	✓ Upgraded	Changed from SS34 (40V) to SS310 (100V). D2 is essential — without it Q2's 40V V_{ds} is instantly exceeded on turn-off. 40V reverse rating left no margin on 16V rail. (The 100 μH used for the turn-off spike is a deliberate worst-case; typical slot motors are 10–50 μH , so the real di/dt overshoot is lower — diode margin intentional.)
D4 MBRS360 (mode C series diode)	Series reverse-block diode (D4) on the current-source injection path. Carries the anti-brake injection current (up to 3A).	3A avg, 60V, 150A peak (single pulse) Same SMB footprint	✓ Upgraded	Changed from SS34 (80A peak, marginal under repetitive 20kHz stress) to MBRS360 (60V, same SMB footprint). SS54 (5A, 150A peak) is equally suitable. D4 position is the most electrically stressed diode in the design.
F1 polyfuse (WHITE rail)	MCU + LDO + gate drivers $\approx 35mA$ total on WHITE rail No motor current on WHITE rail during braking or throttle	Resettable fuse, now 500mA–1A	✓ Corrected	Downrated from 25A to 500mA–1A. F1 protects the MCU/LDO supply on WHITE only. It must NOT be in the BLACK (track) path which carries 40A motor startup and 20A braking current — a polyfuse would trip.
U2 AMS1117-3.3 (LDO)	Load: $\sim 35mA$ $P = (16 - 3.3) \times 0.035 = 0.44W$ at 16V rail $\Delta T = 0.44 \times 62 = 27^\circ C$ (SOT-223, no heatsink)	1A, 125°C T_j , R_{th_ja} 62°C/W	✓ OK	$T_j = 52^\circ C$ at 25°C ambient, 97°C at 70°C ambient. Both within 125°C limit. Solder SOT-223 tab to copper pour for better thermal performance.

Motor startup current of 40A does not flow through any module component during normal operation. Q2 and Q1 are both OFF when the hand controller is delivering power ($V_{BLACK} > 1500mV$). The module is fully passive during throttle. Only during braking does current flow through Q2, and only then does the back-EMF need to be managed.

PRIVATE & CONFIDENTIAL

3.6 User interface — control devices

The module has three physical control devices and a digital display. All are on or attached to the module enclosure. No modification to the hand controller is required.



[Figure 2: User interface controls and display logic — see AART_Brake_UI.svg]

Control	Type	Connector	Function
Mode toggle	3-position switch	PB0 (bit 0), PB1 (bit 1)	Selects Mode A (brake only, all motors — KA floor from Mode C applied if set), Mode B (brushed anti-brake, lookup table), or Mode C (brushless KA tuning, save on leaving). Read each 1ms tick.
Potentiomet	10kΩ linear rotary,	PA3	Mode A: linear CCW→CW brake only

PRIVATE & CONFIDENTIAL

Control	Type	Connector	Function
er	no centre detent	(ADC_IN3)	(DAC_BRAKE_SOFT to DAC_BRAKE_HARD). Mode B: indexes coupled brake+anti-brake table. Mode C: dials KA floor live (CCW=0V, CW=~2V). No centre detent required.
BOOT0 button (J12)	Momentary pushbutton, normally open	PA14-BOOT0 (G0; nBOOT_SE L=0)	TM1637 DIO entry for field firmware update. Not part of normal operation.

3.6.1 What the potentiometer controls in each mode

The pot function depends on the mode selected by the toggle. In all modes, CCW gives minimum output and CW gives maximum:

Mode	Pot function	Output produced
Mode A — Brake only (all motors)	Linear map: CCW = selected soft min (SET BRAKE_OHMS, one of 25/15/5/4/3/2/1Ω), CW = DAC_BRAKE_HARD (dead short). No lookup table.	Q2 brake only. Q1 applies saved_ka_dac as a floor independently (set via Mode C). No anti-brake in Mode A.
Mode B — Anti-brake (brushed only)	Indexes brake_tables[][] and ka_dac_table[] simultaneously. Both brake and anti-brake live from single pot position.	CCW = min brake + max anti-brake (~2V). CW = max brake + no anti-brake. Coupled by table design.
Mode C — KA tuning (brushless, setup mode)	Dials KA floor live: CCW = DAC_KA_OFF (no floor), CW = DAC_KA_MAX (~2V). Display shows KA voltage.	Toggle away from Mode C → 500ms debounce → save saved_ka_dac + LED blinks 3×. Return to Mode A for racing.

Brake ramp-in time and trail release time are UART-only parameters (SET RAMP_MS and SET RELEASE_MS_A/B/C). They cannot be set with the pot. This keeps the pot free for the voltage setpoint function in all modes.

3.6.2 Mode C keep-alive floor procedure

Mode C keep-alive floor procedure:

- 1. Toggle to Mode C. The display switches from track voltage to the live KA floor voltage.
- 2. Rotate the pot to the desired KA floor level. CCW = 0V (no floor). CW = ~2V floor. The display tracks it live.

- 3. Toggle back to Mode A (or Mode B). After 500ms the floor is saved to flash. LED blinks 3×.
- 4. Mode A now applies the saved KA floor. Pot governs brake linearly. Car is ready for brushless racing.

What is saved and how it is applied:

Mode	What is saved	How it is applied
A	saved_ka_dac (KA floor, set by pot on leaving Mode C)	Q1 activates whenever V_BLACK drops below this voltage. 0 (DAC_KA_OFF) = no floor, pure braking. Set via Mode C.
B	No saved value in Mode B	Mode B uses the lookup table directly. Pot CCW = min brake + max anti-brake. No separate saved value.
C	N/A — Mode C is a tuning mode only	Mode C sets the value used in Mode A. It does not apply KA floor itself during racing.

All saved values survive power cycles. They are stored in page 31 of flash (0x08007C00) and loaded at boot. The firmware defaults to DAC_KA_OFF (no injection, no floor) if no valid flash record is found.

3.7 Digital display (TM1637)

A TM1637 4-digit 7-segment LED module connects to header J14, driven directly by the MCU over a 2-wire serial link (CLK on PB4, DIO on PB8). The MCU sends the exact digits to display, so there is no analog proxy and no scaling — the readout shows true values in real time. Specify a TM1637 module with per-digit decimal points (not the colon-only clock variant).

State	Display shows	Source signal
Normal running (Mode A or B)	Track supply voltage (WHITE rail), 0–16V	MCU formats V_WHITE (e.g. 14.2) and clocks it to the TM1637
Mode C active (KA tuning)	Live keep-alive floor voltage, 0–2V	MCU formats the live KA floor (e.g. 0.30) and clocks it to the TM1637

3.7.1 J14 connector pinout (TM1637)

Pin	Signal	Connect to module
1	CLK — TM1637 clock (PB4)	Clock wire to module CLK
2	DIO — TM1637 data (PB8, bidirectional)	Data wire to module DIO
3	GND	Ground wire to module GND

PRIVATE & CONFIDENTIAL

Pin	Signal	Connect to module
4	+3.3V (Vcc)	Power wire to module VCC

The display shows true values in both modes: the WHITE rail voltage (0–16 V, one decimal) while running in Mode A or B, and the live keep-alive floor (0–2 V, two decimals) while tuning in Mode C. Because the MCU drives the digits directly over the TM1637 serial link, no analog scaling or RC filter is involved; the previous PB4-PWM proxy and its TIM3 channel are removed.

4. Firmware Architecture

4.1 File structure

File	Description
Core/Inc/brake_module.h	Constants, pin map, mode/state enums, BrakeCtx_t, public API. Rev 11.
Core/Src/brake_module.c	Full control logic: clock/GPIO/DAC/ADC/DMA/display setup, flash load/save, mode state machine, brake_tick().
Core/Inc/uart_debug.h	DebugParams_t struct, uart_debug public API.
Core/Src/uart_debug.c	UART telemetry, SET/GET command parser, flash SAVE.
Core/Src/main.c	Top-level: SysTick ISR, fault handlers, main loop calling brake_tick().
stm32g051k6.ld	Linker script: 32K flash at 0x08000000, 18K RAM at 0x20000000. Page 31 reserved for NV storage.
Makefile	Docker build: arm-none-eabi-gcc, -DSTM32G051, libopenm3 linked.

4.2 Timing and scheduling

The module uses a polling architecture. `brake_tick()` is called from the main loop as fast as possible. A millisecond counter incremented by SysTick ISR provides internal pacing. ADC conversions run via DMA so the CPU never blocks waiting for a result. All control decisions happen in the 1ms tick context.

4.3 ADC configuration

ADC1 is configured in scan mode with three channels in sequence:

- Rank 1 — PA0 (CH0): WHITE rail voltage sense (R8/R9 divider)

- Rank 2 — PA1 (CH1): BLACK_to_track voltage sense (R10/R11 divider, track side of the pass switch)
- Rank 3 — PA3 (CH3): pot wiper (10k linear, 0–3.3V)

Results are transferred to a 3-element DMA buffer. DMA TC interrupt sets `s_adc_ready` flag. `brake_tick()` reads the flag and processes one set of readings per call. Trigger: software (SWSTART), retriggered at end of each DMA transfer.

4.4 DAC configuration

Both DAC channels are used — DAC1 for the brake gate (Q2), DAC2 for the anti-brake demand (Q1):

- DAC_CH1 (PA4) — Q2 brake gate. 0–3.3V linear, maps from the selected minimum resistance (25/15/5/4/3/2/1 Ω) to dead short at the FET.
- Anti-brake / keep-alive demand is a DC voltage from DAC2 (DAC_OUT2 on PA5), fed through the demand divider and GD3 translator to the GD1 current-source op-amp. The DAC2 code sets the demand (0 = off, full = ~3A); see §7A.7.

Both channels use software trigger mode. `brake_tick()` calls `set_brake_dac()` and `set_ka_dac()` which write to DAC_DHR12R1/R2 and fire DAC_SWTRIGR. The op-amp buffers (GD1, GD2) buffer the DAC output to drive the FET gates in linear mode.

4.5 Key constants

Constant	Value	Description
DAC_BRAKE_OFF	0	Q2 off. 0V gate, $V_{gs} < V_{th}$.
DAC_BRAKE_SOFT	1860	Legacy reference (~1.5V gate, ~8 Ω). Superseded by SET BRAKE_OHMS table selection.
DAC_BRAKE_HARD	3095	Q2 near dead short (~2.5V gate).
DAC_KA_OFF	0	Q1 off (DAC2 code 0 — no anti-brake injection).
DAC_KA_MAX	2480	Q1 maximum conduction (~60.6% duty → ~2V filtered demand → ~3A).
DAC_BRAKE_SOFT	see SET BRAKE_OHMS (idx 0–6, default 4)	Minimum brake resistance is selectable via SET BRAKE_OHMS (snaps to 25/15/5/4/3/2/1 Ω), choosing one of seven stored tables. <code>brake_ohms_idx</code> (0-6) stored in flash. Default index 4 (~3 Ω).
DAC_KA_MAX (anti-brake max)	2480 (DAC_KA_MAX)	Maximum Q1 conduction. Anti-brake level follows pot via <code>ka_dac_table[]</code> .
KA_SAVE_DEBOUNCE_	500	KA save debounce after leaving Mode C. Fixed at

Constant	Value	Description
MS		500ms in firmware.
RAMP_MS_MAX	200	Maximum ramp-in time. SET RAMP_MS 0–200.
RELEASE_MS_MAX	500	Maximum trail release time. SET RELEASE_MS_A/B/C 0–500.
FLASH_MAGIC	0xAA270008	Rev 11. Old records invalidated on first boot.
FW_ID_STRING	AART-BRAKE- v11- STM32G051	Returned by GET ID.

4.6 Control flow

`brake_tick()` runs the full state machine each call. The flow:

- 1. Read ADC DMA buffer — `white_mv`, `black_mv`, `pot_raw`.
- 2. Read toggle. Detect mode transitions — specifically leaving Mode C (triggers 500ms debounce save timer).
- 3. Latvian brake check — WHITE dV/dt . If triggered, apply `DAC_BRAKE_HARD` and return.
- 4. Update `brake_active`: read `I_DIR` (PA6) first — if asserted (reverse current = controller braking), force `brake_active` = true and hold it (`IDIR_HOLD_TICKS`) across brief reverse-detect dropouts; otherwise apply `V_BLACK` hysteresis (`BRAKE_ENTER/EXIT_MV` on `BLACK_to_track`) for coast entry and forward-drive exit.
- 5. Dispatch to mode handler: Mode A (linear brake + KA floor), Mode B (anti-brake table), or Mode C (KA tuning).
- 6. Mode A: linear map `pot` → Q2 brake, `saved_ka_dac` applied as Q1 floor when `V_BLACK` low. Mode B: table lookup for both outputs. Mode C: live `ka_dac_table` preview on the display, linear brake during braking.
- 7. If leaving Mode C and 500ms has elapsed: save `ka_dac_table[pot_idx]` as `saved_ka_dac`, blink LED 3×.
- 8. Update display: track voltage normally; live `ka_dac_table` value while in Mode C.
- 9. Update LED blink state.

5. Bench Testing

5.1 Brake detection verification

Inject a variable DC voltage on the `BLACK_to_track` node via a bench supply or potentiometer. Observe UART telemetry state field:

- `V_BLACK` < 500mV → state transitions to BRAKING (or RAMP_IN if ramp enabled)
- `V_BLACK` > 1500mV → state returns to PASS

PRIVATE & CONFIDENTIAL

- Between 500mV and 1500mV → state holds (hysteresis prevents chatter)

5.2 Power budget

Production target at 4MHz LP Run mode: approximately 524μA total. MCU ~372μA, op-amp buffers ~50μA each, LDO (MCP1700) 2μA quiescent. Hold-up capacitor C_HU 100μF sustains MCU for >300ms after PSU cut. See section 7A.6 for full analysis.

6. Open Items and Known Limitations

Item	Description	Status
Q1 anti-brake current source	Superseded. The anti-brake stage is now a DAC2-set constant-current source (section 7A.7). The floating-supply circuit (ZD1/R_Z1/C_Z1/LM321) is removed.	Resolved by redesign — anti-brake is now a DAC2-set constant-current source (section 7A.7). Remaining bench items: current-source loop stability and pass-device heatsink sizing (~47W transient at stall).
Pass-switch control polarity	INA210/LM311 reverse threshold (~0.16A) and hysteresis, the gate-drive charge pump, and the reverse response time (~2μs) — verify on bench before PCB (section 7A.2).	Open — bench verify.
Latvian brake primary trigger	WHITE dV/dt detects PSU cut and car-out-of-slot events. Implemented in current firmware (STATE_LATVIAN_BRAKE). Threshold configurable via SET LATVIAN_DVDT. Disabled by default (0 = off). The alternative trigger (BLACK braid loss) was considered but WHITE dV/dt covers both PSU cut and out-of-slot and is sufficient.	Implemented. UART-configurable threshold (SET LATVIAN_DVDT). Closed.

7A. Production Architecture — Design Decisions and Analysis

7A.1 Overview

This section documents the key architectural decisions made during the design process, the alternatives considered, and the rationale for each choice.

7A.2 Reverse-blocking pass element — two N-channel bidirectional switch

Design change (this revision). The BLACK controller-input path needs a low-loss element that passes the controller's drive to the track with minimal insertion drop but blocks reverse current (track voltage or motor back-EMF feeding back into the controller output). Earlier options — an NPN level-shift, a P-channel + driver, and an LTC4412/ideal-diode IC — are all retired. The anode-powered ideal-diode-IC approach fails for two reasons specific to this circuit, both identified in review:

- Bootstrap/startup region: an ideal-diode controller draws its power from the anode (the BLACK line). Until BLACK rises above the controller's minimum operating voltage, the controller is unpowered and cannot enhance the FET, so the pass FET's body diode carries the current — ~1V drop at several amps — in exactly the low-throttle region where the racer modulates most.
- Turn-on discontinuity: as BLACK crosses the controller's turn-on threshold, the series drop collapses from ~1V (body diode) to ~20mV (regulated) almost instantly. The motor sees that ~1V step for a tiny trigger movement — a dead/notchy zone that an ISRA-level racer would feel immediately.

7A.2.1 Topology — common-source N-channel pair

Two N-channel MOSFETs are placed in series back-to-back with their sources tied together (common-source). Drains go to the two BLACK terminals (controller side and track side); the common source is the driver reference node. The opposing body diodes mean the switch blocks in both directions when off and conducts with low $R_{ds(on)}$ in both directions when on, so turning it off blocks reverse current. P-channel is deliberately avoided: at 40A its on-resistance (best-in-class ~6–12m Ω at these voltages) would add 0.2–0.5V of drop and 9–20W of heat, defeating the low-drop requirement. N-channel hits sub-milliohm.

7A.2.2 Driver — INA210 + LM311 ideal diode (sense and compare on +5V, gate drive from WHITE)

Design change (this revision). The reverse-block driver is now an ideal diode built from a TI INA210 current-sense amplifier (U5) and an LM311 comparator (U6), replacing the TPS1212-Q1 of the previous revision. The INA210 (gain 200) senses the main-path current across the shunt R_{sns_sw} ; the LM311 compares its output to a small threshold and turns the Q5/Q6 common-source pair off when the current reverses. The INA210 and LM311 run from the +5V analog rail — not the anode — so they are alive regardless of the BLACK voltage, sidestepping the bootstrap problem of anode-powered ideal-diode ICs.

- VS (supply) ties directly to the WHITE rail. Its operating range is 3.5–73V (74V absolute max), so 12–16V is well inside it and no VCC clamp is needed (the LTC7001 required one; this part does not). Control is alive regardless of BLACK voltage — same bootstrap fix as before, fewer parts.
- Why the INA210 + LM311 ideal diode over the TPS1212-Q1: the motivation is the motor-current glitch when the brake contact closes. The TPS1212's ~6–10 μ s reverse response let ~29A flow for ~13 μ s ($\approx 377A \cdot \mu s$) and dragged the motor voltage down enough to risk browning out and restarting the brushless eCom — the very thing the keep-alive exists to prevent. The faster ideal diode (~2 μ s, ~0.16A trip, ~50mV hysteresis) cuts the glitch to ~13A for ~2.2 μ s ($\approx 29A \cdot \mu s$, ~13 $\times$ lower energy) and a much smaller voltage dip. The cost is the TPS1212's integrated protection — I²t overcurrent, 5 μ s short-circuit, IMON, FLT — which the ideal diode does not provide. This is an accepted open item: none of the other stages (brake, anti-brake) carry protection either and the reverse event is brief, but the pass FETs lose their own fault protection. A second open item is the gate drive: the common-source N-FET gates must be lifted above the 0–16V BLACK node — the TPS1212 did this with an internal charge pump, so the ideal diode needs a discrete high-side gate driver / charge pump fed from WHITE and gated by the LM311.

Because the LM311 turns the whole Q5/Q6 pair off on a reverse event, the opposing body diodes then block in both directions — so the same two FETs give true bidirectional reverse blocking. A single FET

could not (its body diode would conduct the blocked direction), which is the reason the back-to-back pair is retained.

7A.2.3 Reverse-current sensing, trip threshold, and response speed

The INA210 senses main-path current across the shunt R_{sns_sw} and presents a ground-referenced voltage (gain 200) to the LM311. The LM311 compares this against a threshold set just above the zero-current point and asserts its reverse-detect output when current flows in reverse; that output both turns the gate driver off (FETs block) and feeds the MCU brake-detect on PA6. With $R_{sns_sw} \approx 1\text{m}\Omega$ the threshold is $\sim 0.16\text{A}$ — INA210 $\pm 35\mu\text{V}$ offset plus the LM311's 50mV hysteresis $\div 200 \approx \pm 160\mu\text{V}$, $\div 1\text{m}\Omega$ — versus $\sim \pm 0.6\text{A}$ for the TPS1212's $\pm 2\%$ -of- 30mV monitor.

7A.2.3.1 Reverse-response speed and the eCom-brownout motivation

The driver behind this whole change is the motor-voltage glitch when the brake contact closes mid-run. With the TPS1212's $\sim 6\text{--}10\mu\text{s}$ reverse response the brake-contact closure let a $\sim 29\text{A}$ spike flow for $\sim 13\mu\text{s}$ and pulled the motor voltage down noticeably — short in human terms, but enough to risk the brushless eCom seeing a brownout and restarting. The INA210 + LM311 ideal diode reaches $\sim 2\mu\text{s}$ in simulation, cutting the spike to $\sim 13\text{A}$ for $\sim 2.2\mu\text{s}$ and the voltage dip with it, roughly $13\times$ less glitch energy. Two caveats on the speed: the $\sim 2\mu\text{s}$ figure must be confirmed with the real INA210 and LM311 models (the INA210 at gain 200 is bandwidth-limited and zero-drift, so it trips on the rising edge rather than at full settling), and the INA210's 26V common-mode ceiling — versus the TPS1212's 73V — must be checked against any track transient at the shunt node.

7A.2.3.2 Sense resistor and insertion drop

The INA210 wants a defined shunt R_{sns_sw} . Sizing it at $\sim 1\text{m}\Omega$ gives $\sim 40\text{mV}$ at 40A and the $\sim 0.16\text{A}$ reverse threshold above; combined with the two FETs ($2 \times 0.59\text{m}\Omega \approx 47\text{mV}$) the total insertion drop is $\sim 87\text{mV}$ at 40A . R_{sns_sw} is a drop-vs-resolution knob: a smaller shunt lowers the drop but coarsens the threshold. The lossless V_{ds} -sense of the previous revision is not available — the INA210 needs the real shunt — and the bidirectional IMON readback of the TPS1212 is gone, though the INA210's own output is an analog current signal that can be routed to a spare ADC if a readback is wanted.

Note: The INA210 is a zero-drift current-sense amplifier (gain 200, common-mode -0.3 to 26V , $\pm 35\mu\text{V}$ offset); the LM311 is a fast bipolar comparator. Watch the INA210/LM311 footprints and pinouts, the Kelvin connection at R_{sns_sw} , the LM311 hysteresis network, and the gate-drive charge pump (open item). Bench-verify the reverse trip threshold, the reverse-detect-to-off time ($\sim 2\mu\text{s}$ target), and the 26V common-mode headroom at 40A before PCB. The LM311 output to the gate driver must be fast and fail-safe (reverse $\rightarrow$ off).

7A.2.4 Device selection and insertion-drop budget (40A)

- Q5, Q6 — 40V N-channel, very low $R_{ds(on)}$. Candidates: ST STL059N4S8AG ($0.59\text{m}\Omega$, PowerFLAT 5×6) or Vishay SiJK140E ($0.34\text{m}\Omega$, PowerPAK 10×12). Two in series.
- Insertion drop at 40A : $2 \times 0.59\text{m}\Omega \approx 47\text{mV}$ across the FETs, plus the shunt R_{sns_sw} ($\sim 1\text{m}\Omega \approx 40\text{mV}$), for $\sim 87\text{mV}$ total. Higher than the bare-FET 47mV , the price of the reverse sense. With Vishay SiJK140E ($0.34\text{m}\Omega$) the FET part drops to $\sim 27\text{mV}$.

Note: At sub-milliohm, PCB copper resistance is a real fraction of the total path. A few cm of 1oz copper is itself $\sim 0.5\text{m}\Omega$, so the insertion-drop budget must include the layout: wide pours, short runs, and Kelvin-sensed comparison points. Budget the whole series path (connector + braid + FET pair + copper) against the target, not just the silicon. Bench-verify the drop at full 40A and confirm comparator turn-off speed prevents reverse current.

7A.3 Linear mode FET control

Both Q1 and Q2 operate in their linear (triode) region rather than as switches. This eliminates PWM switching noise on the braid, which is critical for brushless eCom compatibility. It also removes 20kHz conducted emissions that would complicate CE marking. See section 11.0 for full analysis.

7A.4 Hold-up capacitor sizing

A 100 μ F SMD electrolytic on the 3.3V rail provides ~300ms MCU hold-up after PSU cut at 524 μ A production load. This ensures the firmware maintains the full programmed brake through the power-off transient. See section 7A.6 for worked calculation.

7A.5 Production MCU: STM32G051K6U6

- QFN-32 — provides all required GPIO and DAC peripherals.
- 12-bit DAC (PA4/PA5) — required for linear gate voltage control.
- 32KB flash, 18KB RAM, 64MHz Cortex-M0+ — adequate for Rev 11 firmware.
- Available at JLCPCB. Not a special-order part.

7A.6 Power-down, brownout and hold-up capacitor

7A.6.1 The problem: braking must survive PSU power-off

A conventional brake resistor continues to brake the car when the PSU cuts because it requires no power. An active module loses MCU power at PSU cut. The pull-up on GD2 provides a hardware safe default (Q2 enters braking region), but the hold-up capacitor extends firmware-controlled braking through the transient.

7A.6.2 STM32G051 power supply supervisor: POR, PDR and BOR

Circuit	Function	Threshold	Behaviour on VDD falling
POR (Power-On Reset)	Holds MCU in reset during power-up	VPOR $\approx$ 1.7V (rising)	Not relevant on falling edge
PDR (Power-Down Reset)	Resets MCU when VDD falls below minimum	VPDR $\approx$ 1.6V (falling)	MCU reset. DAC outputs collapse.
BOR (Brownout Reset)	Programmable earlier warning threshold via option bytes	VBOR1–VBOR4: 1.7V–2.8V selectable	MCU reset when VDD < VBORx

7A.6.3 Why the STM32G051 has suitable power consumption

Component	Prototype (64MHz)	Production (4MHz)	Notes
STM32G051 MCU	~6mA	~372µA	G0 family: ~93µA/MHz. 4MHz × 93 = 372µA.
Gate drivers GD1/GD2	~0.05mA (op-amp buffers)	~50µA (op-amp)	Dominant saving. Linear mode op-amps replace switching drivers.
LDO	~5mA (AMS1117)	~2µA (MCP1700)	AMS1117 quiescent is 5mA. MCP1700 is 2µA.
ADC, DAC, GPIO	~500µA	~100µA	ADC scanning 3 channels at 1ms interval.
Total	~21.5mA	~524µA	Production well under 1mA.

7A.6.4 Hold-up capacitor sizing — worked calculation

$C = I \times t / \Delta V$, where $I = 524\mu\text{A}$, $t = 100\text{ms}$, $\Delta V = 3.3 - 1.7 = 1.6\text{V}$:

- $C = 0.000524 \times 0.1 / 1.6 = 32.8\mu\text{F}$ minimum
- Use 100µF (3× margin). Verification: $t = 100 \times 10^{-6} \times 1.6 / 0.000524 = 305\text{ms}$.
- 3× margin accounts for X5R/X7R capacitance derating at DC bias (~40% typical), age degradation, and load estimate uncertainty.

7A.6.5 Component selection: 100µF 6.3V SMD electrolytic

Parameter	Value	Rationale
Capacitance	100µF	>300ms hold-up at 524µA, 3× margin
Voltage rating	6.3V min, 10V preferred	Reduces derating on 3.3V rail
Package	SMD electrolytic or 1210 polymer	Polymer SMD has lower ESR
Stored energy	$E = \frac{1}{2}CV^2 = 0.54\text{mJ}$	Powers MCU only. Well below any regulatory stored-energy threshold.

The 100µF hold-up cap is in addition to the ST-recommended 100nF + 4.7µF decoupling at VDD pins.

7A.6.6 Behaviour on power-down

Time	Event	MCU state	Brake output
t = 0	PSU switched off.	Running normally	Programmed brake at pot-set level

PRIVATE & CONFIDENTIAL

Time	Event	MCU state	Brake output
	WHITE rail begins to fall.		
$t \approx 0\text{--}10\text{ms}$	LDO ceases to regulate. C_HU supplies VDD.	Running from capacitor	DAC maintains programmed brake
$t \approx 0\text{--}305\text{ms}$	VDD falls from 3.3V toward 1.7V as C_HU discharges.	Running. ADC less accurate below 2V.	Programmed brake maintained
$t \approx 305\text{ms}$	VDD < PDR threshold. MCU resets. DAC collapses.	Reset	Pull-up circuit takes over
$t > 305\text{ms}$	R2 pull-up holds Q2 gate in braking region.	No power	Hardware safe-default brake

7A.7 Anti-brake stage — constant-current source (DAC2-set, 3A)

Design decision (this revision). The anti-brake demand is generated by DAC2 (DAC_OUT2 on PA5), not a filtered PWM. Earlier revisions used a 50 kHz TIM2_CH1 PWM plus an RC filter (C_KA) on the same pin, which freed DAC2 but left it unused. DAC2 delivers clean 12-bit DC straight into the demand divider and GD3 translator: it deletes C_KA and its series R, removes a switching node, and makes both analogue outputs DAC writes (DAC1 = brake gate, DAC2 = anti-brake demand) — finally using the dual DAC the STM32G051 was chosen for (§7C). The divider and GD3 are unchanged; they are required because GD1 is WHITE-referenced.

Design change (this revision). The anti-brake stage is a constant-current source, not the voltage follower of the previous revision. Real transistor controllers (Kobic and similar) implement anti-brake as a current source — typically 2.5–3A at maximum, set by the pass transistor's gain — not a fixed voltage. The follower was wrong for the case that matters most: a car stationary on the start line presents a near short-circuit, so a voltage follower holding ~2V into it would drive an unbounded current (tens of amps, limited only by the device SOA). A current source bounds the injection to ~3A regardless of motor impedance, which is both the correct behaviour and what a racer's muscle memory expects.

7A.7.1 Topology (per the SIMetrix design)

A classic high-side op-amp current source. The op-amp drives the pass device and regulates the voltage across a sense resistor in the supply leg to equal the demand, so the load current is held constant at $I = V_{\text{demand}} / R_{\text{sense}}$ independent of the motor:

- Op-amp supply — CRITICAL: GD1 is powered from the WHITE rail (~12–16V), NOT the +5V rail. Because the sense resistor sits in the WHITE leg, the op-amp's inputs and output sit at a common mode near the rail (~16V); a 5V op-amp cannot sense or drive there. GD1 must also be rail-to-rail INPUT, because it senses right at V+ — an LM358/LM321-class part will NOT work (its input common-mode range stops ~1.5V below V+ and cannot reach the rail). Use the TI LM7321 (RRIO, 2.5–32V, SOT-23); for higher precision the OPA192 is an alternative. The op-amp output swings near WHITE to drive the TIP147 base. Only the GD2 brake buffer remains on the +5V rail; unlike GD1 it needs rail-to-rail OUTPUT (not input) and a gain of ~1.5 to lift the 3.3V DAC to a ~5V gate drive — the same LM7321 covers it (budget alternative for GD2 only: TLV9061 — which is RRO but not rail-to-rail input, so it cannot serve GD1). The +5V supply also conveniently

caps the gate at ~5V — full-on for the low-R_{ds} FET and safe for a logic-level gate. Both are the same LM7321 type but two separate singles, since GD1 (WHITE) and GD2 (+5V) sit on different rails; a dual LM7322 could combine them only if both ran on one rail.

Note: This corrects an earlier revision that placed GD1 on +5V. It is confirmed by the SIMetrix design (AntibrakeConstCurrent.wxsch), where the op-amp V+ ties to the supply rail (V3) and it senses just below it.

- R_{sense} = 0.1Ω in the WHITE-rail leg of the pass device. At 3A that is 0.3V across it (0.9W).
- Demand is a DC voltage from DAC2 (DAC_OUT2 on PA5), divided to a 0.30V ground-referenced reference (R_KA1 6.8k / R_KA2 1.2k), then translated to the WHITE rail by GD3 (a unity-gain difference amplifier) so the current-source demand is WHITE – 0.30V. The MCU sets the DAC2 code from the single control-pot reading (PA3 → ka_dac_table[]); there is no second pot and no pot in the current-source loop itself. Code 0 → 0A (off); full → 3A.

Worked ceiling (DAC count → current): with R_KA1 = 6.8 kΩ and R_KA2 = 1.2 kΩ the demand divider $R_{KA2}/(R_{KA1}+R_{KA2}) = 1.2/8.0 = 0.15$ scales the DAC2 output, so $I_{max} = (DAC_KA_MAX/4095 \times 3.3\text{ V}) \times 0.15 / R_{sense} = (2480/4095 \times 3.3\text{ V}) \times 0.15 / 0.1\ \Omega = 2.0\text{ V} \times 0.15 / 0.1\ \Omega = 0.30\text{ V} / 0.1\ \Omega = 3.0\text{ A}$. GD3, a unity-gain difference amplifier, then translates this 0.30 V demand to the WHITE-side node (WHITE – 0.30 V) that GD1 regulates.

- Pass device: TIP147 PNP Darlington (TO-220 + heatsink) as the primary choice — a bipolar pass element is robust in linear operation. An N-channel MOSFET (SUP70060E, or RFP50N06 / FDP61N20 / SUP50020EL) is the documented alternative; at our low V_{ce}/V_{ds} (≤16V) the SOA is power-limited, so either works.

The current setpoint is generated by DAC2 as clean DC, feeding the demand divider and GD3 translator. The op-amp and R_{sense} form the loop; the pass device supplies the current. The injection still goes through D4 (MBRS360) into the motor node, blocking reverse current from the track.

7A.7.2 Why a current source is also safer

- Shoot-through protection: if a fault turns the brake (Q2) and anti-brake on together, a current source caps its contribution at ~3A instead of feeding a near-short — a genuine fault-tolerance property the follower lacked.
- Add open/short detection on the current-source loop itself, plus a fast over-current trip, to guard against a component failure in the source turning it into an uncontrolled path.

7A.7.3 Firmware impact

Anti-brake demand is a DAC2 code, written to DAC_DHR12R2. The ka table maps pot index → DAC code (0 = off ... full = 3A); polarity is 0 = off, so the "KA off = 0" convention holds. DAC1 drives the brake gate (linear gate voltage for resistance control); DAC2 drives the anti-brake demand. The DAC_KA_OFF/DAC_KA_MAX constants are now literal DAC codes.

7A.8 Thermal — anti-brake pass device (worst case at stall)

At stall the motor is ~0V, so the pass device drops nearly the full rail at the set current. Using the steady-state SOA method ($T_j = T_a + P_d \cdot (R_{\theta jc} + R_{\theta cs} + R_{\theta sa})$) from the SIMetrix design, with V_{supply}(max) = 16V and T_a = 30°C. Past the junction there are two series terms beyond R_{θjc} (junction-to-case, datasheet): R_{θcs}, the case-to-sink thermal-interface material (§7A.8.1), and R_{θsa}, the heatsink's sink-to-ambient resistance. Their sum is the case-to-ambient R_{θca} = R_{θcs} + R_{θsa}, so the heatsink actually specified is R_{θsa} = R_{θca} – R_{θcs}, not R_{θca} itself:

- At I_{out} = 2.75A → P_d ≈ 44W (no tolerance). TIP147: SOA 125W at T_c=25°C, allow T_j=125°C, R_{θjc}=1°C/W → required R_{θca} = (125–30)/44 – 1 = 1.16°C/W, giving T_c ≈ 81°C.

- SUP70060E (MOSFET): SOA 200W, allow $T_j=150^\circ\text{C}$, $R_{\theta jc}=0.75^\circ\text{C/W} \rightarrow R_{\theta ca} = (150-30)/44 - 0.75 = 1.98^\circ\text{C/W}$, $T_c \approx 117^\circ\text{C}$. The MOSFET allows a smaller heatsink but runs hotter (or the same heatsink at a cooler T_j).
- At the full 3A demand the worst case rises to $P_d \approx 47\text{W}$, tightening the TIP147 case-to-ambient budget to $R_{\theta ca} \approx 1.0^\circ\text{C/W}$. With a realistic grease interface ($R_{\theta cs} \approx 0.5^\circ\text{C/W}$ for a non-isolated TO-220, §7A.8.1) the heatsink itself must then be $R_{\theta sa} \approx 0.5^\circ\text{C/W}$ — roughly twice the sink the bare $R_{\theta ca}$ figure implies. Design the heatsink to the 3A figure with the interface included.

7A.8.1 Case-to-heatsink interface (thermal interface material)

The case-to-ambient term above is not the heatsink alone: it is the interface $R_{\theta cs}$ in series with the heatsink $R_{\theta sa}$. The interface path is case $\rightarrow$ contact resistance $\rightarrow$ bulk TIM $\rightarrow$ contact resistance $\rightarrow$ sink, and on a small package the two contact resistances often outweigh the bulk. Omitting $R_{\theta cs}$ is optimistic — against the $\sim 1^\circ\text{C/W}$ case-to-ambient budget an interface of $0.3\text{--}0.5^\circ\text{C/W}$ is 30–50% of the total.

The bulk part alone is $R_{\theta \text{ bulk}} = L / (\lambda \cdot A)$: L the bond-line thickness (m), λ the TIM conductivity ($\text{W/m}\cdot\text{K}$), A the contact area ($\approx 90\text{ mm}^2$ for a TO-220 tab, $\approx 150\text{ mm}^2$ for TO-247). Two cautions. L is not a value you set — it is whatever surface flatness, mounting pressure and paste rheology leave, typically 25–100 μm and unknown without measuring. And the formula ignores the two contact resistances, so it underestimates badly: for ordinary grease it returns roughly a quarter of the real figure ($\lambda=4\text{ W/m}\cdot\text{K}$, $L=50\text{ }\mu\text{m}$, $A=90\text{ mm}^2 \rightarrow \sim 0.14^\circ\text{C/W}$ versus a measured $\sim 0.5^\circ\text{C/W}$).

So do not size the interface from λ . Use the TIM datasheet's measured $R_{\theta cs}$ / thermal-impedance where given — it already includes the contact resistances, though it is quoted at a stated mounting pressure and worsens at lower pressure. Otherwise take a conservative tabulated value for the package-plus-TIM, or measure it: drive a known P_d and read $R_{\theta cs} = (T_c - T_s)/P_d$.

Representative $R_{\theta cs}$ for a non-isolated TO-220 (no mica or insulating pad), mounted with pressure: dry $\approx 1.0\text{--}1.2$ (never run dry); ordinary silicone grease ≈ 0.5 ; high-conductivity grease ($\lambda \approx 3\text{--}5\text{ W/m}\cdot\text{K}$) ≈ 0.3 ; phase-change film (e.g. Honeywell PTM7950, $\lambda \approx 8.5\text{ W/m}\cdot\text{K}$) $\approx 0.2^\circ\text{C/W}$. An electrically-insulating interface would add roughly $+0.3$ to $+1^\circ\text{C/W}$, but the TIP147 tab carries the collector and is not isolated here, so a bare grease or phase-change interface is correct. A TO-247 part, with more contact area, roughly halves these.

Folded into the 3A worst case ($P_d \approx 47\text{W} \rightarrow R_{\theta ca} \approx 1.0^\circ\text{C/W}$): ordinary grease ($R_{\theta cs} 0.5$) needs a heatsink of $R_{\theta sa} \approx 0.5^\circ\text{C/W}$; a good phase-change film ($R_{\theta cs} 0.2$) relaxes it to $R_{\theta sa} \approx 0.8^\circ\text{C/W}$ — a real change in sink size. Phase-change films also apply more cleanly than paste and, reflowed once under spring pressure, give a thin repeatable bond-line, preferable for production if a current part is sourced. Note the interface sits in the junction's fast path ($R_{\theta jc} + R_{\theta cs}$) ahead of the sink mass, so it raises peak T_j directly and is not hidden by the burst-energy argument below.

These are steady-state numbers and therefore conservative. Anti-brake is active only in short bursts, so the heatsink's thermal capacitance can be factored in: for a burst much shorter than the sink's thermal time constant, the rise is set by capacitance, $\Delta T \approx \text{Energy} / C_{th}$, not by $R_{\theta ca}$. A $\sim 50\text{g}$ aluminium sink has $C_{th} \approx 0.05\text{kg} \times 900\text{ J/kg}\cdot^\circ\text{C} \approx 45\text{ J/}^\circ\text{C}$, so a 2s, 47W burst ($\approx 94\text{J}$) lifts it only $\sim 2^\circ\text{C}$ before it cools between launches. So the practical heatsink can be smaller than the steady-state $R_{\theta ca}$ implies — size it from the worst-case burst energy and the racing duty cycle, not continuous 47W.

Note: Both the brake FET (Q2 (TW100N03CC), section 7A.8 prior content folded here) and the anti-brake pass device run linear and need heatsinks. Confirm the actual anti-brake active time per lap on the bench to set the heatsink via the pulse-energy method above.

7A.9 USB-C interface — UART bridge and auxiliary power

A USB-C connector is added to provide two functions: a UART terminal connection to a laptop or PC for issuing SET/GET commands and monitoring telemetry, and auxiliary power to the module when the track is not connected (bench configuration, parameter setting before a race).

7A.9.1 New components

- J_USB — USB-C receptacle, 16-pin SMD (HRO TYPE-C-31-M-12 or equivalent, JLCPCB). Carries VBUS, GND, D+, D-, CC1, CC2. Mid-mount or top-mount.
- U3 — CH340N USB-UART bridge, SOP-8 (JLCPCB C506813, ~€0.15). Internal 12MHz clock, no crystal required. Supports 3.3V supply.
- R_CC1, R_CC2 — 5.1kΩ 0402 each, CC1 and CC2 to GND. Required for USB-C DFP detection (signals to host that this is a UFP device and requests 5V/0.9A).
- D_USB — BAT54 Schottky diode, SOT-23 (or equivalent, JLCPCB). VBUS in to LDO input, prevents reverse current from track WHITE rail reaching the USB host.
- F_USB — 500mA polyfuse 0805, in VBUS line before D_USB. Protects USB host from shorts on the module.
- C_USB1 — 10μF 10V MLCC 0805, VBUS decoupling.
- C_USB2 — 100nF 0402, CH340N VCC decoupling.

7A.9.2 Power supply arrangement

The module has two power inputs: the track WHITE rail (12–16V) and USB VBUS (5V). Both feed the existing LDO (U2) input through protection diodes:

- WHITE rail → F1 (polyfuse, existing) → D_WHITE (SS310 Schottky, existing) → LDO input
- VBUS → F_USB (polyfuse, new) → D_USB (BAT54 Schottky, new) → LDO input

Both diodes OR into the LDO input. The higher voltage dominates: WHITE (12–16V) always has higher voltage than VBUS (5V) so WHITE powers the LDO when both are connected. The LDO output is 3.3V in both cases.

USB host protection: D_USB blocks the WHITE rail voltage from reaching the USB host. The USB host sees only its own VBUS through F_USB. There is no path from WHITE to VBUS. This is the critical isolation requirement and it is met by the series Schottky.

Note: When only USB is connected (no track), VBUS supplies the LDO at 5V. The LDO regulates to 3.3V normally. The module is fully functional for UART configuration. The FET gate drivers and FETs are not powered (WHITE = 0V) but the MCU and UART run normally.

7A.9.3 CH340N wiring

The CH340N is configured for 3.3V operation (V3 pin shorted to VCC, both at 3.3V):

- VCC (pin 8) → 3.3V rail
- V3 (pin 1) → VCC (shorted, enables 3.3V I/O mode)
- GND (pin 2) → GND
- D+ (pin 3) → USB-C D+
- D- (pin 4) → USB-C D-

PRIVATE & CONFIDENTIAL

- RXD (pin 5) → MCU PA9 (USART1_TX). CH340N receives what MCU transmits.
- TXD (pin 6) → MCU PA10 (USART1_RX). CH340N transmits to MCU.
- Unused pins (7, 8) left unconnected or tied per datasheet

Note: CH340N batch numbers > B40 include internal backflow prevention. For earlier batches, a series Schottky on RXD prevents current injection into the MCU when the MCU is unpowered but USB is connected.

7A.9.4 USB-C connector wiring

- VBUS (pins A4, B4) → F_USB → D_USB → LDO input
- GND (pins A1, B1, A12, B12) → GND
- D+ (pins A6, B6) → CH340N pin 3
- D- (pins A7, B7) → CH340N pin 4
- CC1 (pin A5) → R_CC1 (5.1kΩ) → GND
- CC2 (pin B5) → R_CC2 (5.1kΩ) → GND
- SBU1, SBU2, SSTX, SSRX: not connected (USB 2.0 only)

7A.9.5 Driver installation

The CH340N requires a driver on the host computer. On Linux and macOS (10.9+) the ch341 kernel module is included and loads automatically. On Windows 10/11 the driver is available from the WCH website (www.wch-ic.com). Once installed the module appears as a standard COM port (Windows) or /dev/ttyUSBx (Linux) or /dev/tty.wchusb* (macOS). Any serial terminal at 115200 8N1 can then issue UART commands.

7A.10 Configurable minimum brake resistance

The minimum braking resistance (the resistance at pot CCW, the gentlest brake setting) is motor-dependent and spans a wide range. A small brushless motor might need as little as 1Ω, while a conventional brushed motor used for scale-model racing might want as much as 25Ω. The original hardcoded ~8Ω (DAC_BRAKE_SOFT = 1860) suited neither end of that range well. This value is now selectable at runtime — the 25/15/5/4/3/2/1Ω table via SET BRAKE_OHMS — without being hardware-dependent.

7A.10.1 Why stored tables, not runtime scaling

Two implementations were considered. The first rescaled a single base table on every control tick to move its soft endpoint. The second — adopted here — stores five precomputed tables, one per nominal resistance, and selects one by index. The stored-table approach was chosen because it performs zero arithmetic on the table in the control loop: the 1ms tick is a pure array lookup, keeping timing fully deterministic and avoiding any floating-point or division on the Cortex-M0+ (which has no FPU).

7A.10.2 The seven tables

Seven brake tables are stored, each a gamma-1.8 curve from its soft endpoint up to DAC_BRAKE_HARD (dead short). The resistances are spaced non-linearly (25, 15, 5, 4, 3, 2, 1Ω — coarse at the gentle end,

dense at the firm end where small changes matter most) so the steps are finer at the firm/low-ohm end where small changes matter most, and coarser at the gentle end. Higher DAC = higher Q2 gate voltage = lower Rds(on) = lower resistance, so lower-ohm settings use higher soft endpoints:

- Index 0 — ~25Ω full scale (gentlest minimum brake; e.g. hardbody homeset 1/32)
- Index 1 — ~15Ω full scale
- Index 2 — ~5Ω full scale
- Index 3 — ~4Ω full scale
- Index 4 — ~3Ω full scale (default)
- Index 5 — ~2Ω full scale
- Index 6 — ~1Ω full scale (firmest minimum brake)

Mode B (anti-brake) indexes the selected table directly. Mode A and Mode C (linear maps) use the selected table's soft endpoint as the CCW value of map_u16. The anti-brake (ka_dac_table) is unaffected — it is a voltage injection, independent of brake resistance.

7A.10.3 UART command and storage

- SET BRAKE_OHMS <ohm> takes the requested resistance and snaps to the nearest stored table (25/15/5/4/3/2/1Ω). Takes effect immediately; SAVE persists it.
- The selected index (0-6) is stored in flash DW2 (replacing the former brake_soft_dac slot).
- GET reports the current setting as brake_ohms=N.
- Default on first boot or after flash erase: index 1 (~3Ω).

7A.10.4 Calibration

Note: The 1-5Ω labels are nominal estimates. True effective resistance at a given gate voltage depends on the specific FET, its temperature, and the instantaneous braking current, and cannot be computed in firmware. The per-step soft DAC endpoints are deliberately left as calibration placeholders in this revision. The earlier values were anchored to an 8Ω≈1860 estimate that proved far too low (25Ω brake pots are used for hardbody homeset 1/32 cars), so every endpoint must be measured on the bench with the target FET (IRFP250N) and motor and then committed via tools/gen_brake_tables.py.

To calibrate: select each setting with SET BRAKE_OHMS, measure or feel the actual braking behaviour with the target motor under load, and if the endpoints need adjustment, edit OHM_LABELS / SOFT_ENDPOINTS in tools/gen_brake_tables.py, regenerate the tables, and rebuild the firmware. This keeps the runtime path free of any table arithmetic while allowing precise per-motor tuning.

7A.9.6 Firmware update over USB-C

No custom bootloader is required to update firmware over the USB-C connection. The STM32G051 contains a factory bootloader in system ROM that cannot be erased and supports firmware loading over USART1 (PA9/PA10) — the same pins the CH340N bridge is wired to. A single USB-C cable to a host PC is therefore sufficient to reflash the module.

Important distinction: this is the ROM UART bootloader reached through the USB-to-UART bridge, not native USB DFU. The STM32G051 has no USB device peripheral (USB appears only on the larger G0B1/G0C1 parts), so the module does not enumerate as a DFU device. From the host side the workflow is identical to any serial flash: open the CH340N COM port and use stm32flash or STM32CubeProgrammer in UART mode.

7A.9.7 CH340N reset limitation and update procedure

The CH340N is the minimal 8-pin variant and does not expose the DTR/RTS modem-control lines that auto-reset circuits use. The bootloader entry therefore cannot be automated from the flashing tool; it is performed manually with the BOOT0 (J12) and NRST (J13) buttons already present in the design:

- Connect the USB-C cable to the host PC. The module appears as a COM port (/dev/ttyUSB* on Linux, /dev/tty.wchusb* on macOS).
- Hold BOOT0 (J12) high, press and release NRST (J13), then release BOOT0. The MCU starts in the TM1637 DIO, listening on USART1.
- On the host, flash the image: `stm32flash -w aart-brake.bin -v /dev/ttyUSB0` (or STM32CubeProgrammer in UART mode). Baud is auto-detected from the 0x7F sync byte.
- Press and release NRST again (BOOT0 now low). The new firmware runs.

Note: If automatic, button-free updates are ever required, a small application bootloader could be added that is triggered by a UART command (e.g. ENTER_DFU), receives the new image, and jumps to it. This costs flash, a partitioned memory map, and the usual brick risk of a self-update path, so it is not used here — the ROM UART bootloader with manual BOOT0/NRST is the lower-risk standard route for occasional updates.

Note: Before PCB commit, confirm against ST application note AN2606 that the TM1637 DIO USART maps to PA9/PA10 on the specific STM32G051 variant used. This holds for the G0x1 family but is worth a quick verification, as a mismatch would prevent the CH340N bridge from reaching the bootloader.

7A.10.5 How brake and anti-brake are encoded in the table

In Mode B a single pot position drives two outputs simultaneously. The coupling is purely positional: the pot reading is reduced to one 8-bit index, and that index reads two parallel 256-entry tables at the same time — one for the brake FET (Q2), one for the anti-brake / keep-alive FET (Q1).

The decode runs every 1ms tick and contains no arithmetic or interpolation:

```
idx      = pot_raw >> 4;                // 12-bit ADC -> 8-bit index
brk_dac  = brake_tables[active_ohms_idx()][idx]; // Q2 gate drive
ka_dac   = ka_dac_table[idx];            // Q1 anti-brake demand (DAC2)
```

Both stored values are 12-bit DAC codes (0–4095), i.e. gate-drive levels that become gate voltages through the op-amp buffers. They are not ohms or volts directly — the mapping from a DAC code to an actual resistance or injection voltage is FET-, temperature- and current-dependent and must be calibrated on the bench.

The two tables are built to sweep in opposite directions, so rotating the pot trades one output for the other:

- Index 0 (pot full CCW): brake at its soft minimum, anti-brake at maximum (ka_dac $\approx$ 2480, ~2V forward injection).
- Index 255 (pot full CW): brake at dead short (3095), anti-brake off (demand = 0, Q1 off — DAC2 current source).
- Higher brake DAC = higher Q2 gate voltage = lower R_{ds} = more braking. Higher anti-brake DAC = more injection. The two are applied in different phases — Q2 brakes while the trigger is in the brake zone, Q1 injects while running (after brake exit) — both indexed by the same pot position, so one pot sets the balance.

The brake curve uses a gamma-1.8 shape giving fine resolution at the soft end; the anti-brake curve fades to zero before full CW (by default at about 65% of pot travel) so the firmest part of the brake range carries no injection at all.

7A.10.6 Are separate tables per resistance necessary?

The five stored brake tables currently share the same gamma shape and differ only in where their soft (CCW) endpoint sits. Strictly, that is redundant: the shape is one table and the per-resistance part is just five endpoint values. Two factorings are valid:

- Five full tables (current design): ~2.5KB of flash (brake 2560 B + shared anti-brake 512 B + labels/endpoints 20 B ≈ 3.1KB). Buys zero runtime math and the freedom to give each resistance a different curve shape later.
- One shape + five endpoints, scaled at runtime: ~512 B plus ten endpoint values, at the cost of one multiply and one divide per tick in Mode B. Trivial on the M0+, but it reintroduces the runtime table arithmetic the stored-table design was chosen to avoid.

Because flash is not tight (32KB total, ~12KB used) and there is a plausible future need to give the low-ohm brushless profiles a distinct brake or anti-brake feel, the five-table form is retained. The redundancy is only redundancy while every profile keeps the same shape — the moment any profile diverges (a sharper brake curve, an earlier anti-brake fade), the separate tables stop being redundant. If all profiles are guaranteed to stay identical in shape, the single-shape-plus-endpoints form is the more honest choice.

Per-profile anti-brake tables are also supported (one `ka_dac_table` per resistance instead of a shared one), raising the cost to about 5.1KB — only worth it if the injection profile must differ by motor type.

7A.10.7 Table generation and inspection tooling

Two Python tools accompany the firmware (in `firmware/tools/`). They are the authoritative way to regenerate the tables after bench calibration:

- `gen_brake_tables.py` — builds the C source (`brake_ohms_label[]`, `brake_soft_dac_by_ohm[]`, `brake_tables[][]` and the anti-brake table). Each resistance is described by a Profile with independent knobs for both curves: `brake_soft` and `brake_gamma` shape the brake sweep; `ka_max`, `ka_gamma` and `ka_zero_at` shape the anti-brake sweep and set where injection fades to zero. Run with `--per-profile-ka` to emit independent anti-brake tables. The script prints the exact flash cost of the chosen configuration.
- `test_brake_tables.py` — reproduces the firmware decode (`pot >> 4 -> index -> two lookups`) and reports the brake and anti-brake DAC values at any pot setting, with rough physical estimates. Use `--ohms N --pot P` for a single point or `--sweep` for the full travel.

Calibration workflow: select a profile, measure the real braking and injection behaviour with the target motor under load, adjust that profile's endpoints or gamma in `gen_brake_tables.py`, regenerate, and rebuild. This keeps the runtime path a pure lookup while allowing precise per-motor tuning.

Note: The ohm and injection-voltage figures printed by `test_brake_tables.py` are interpolations from hand-set anchor points, not measurements. The DAC codes are exact; the physical readouts are orientation only until calibrated on the bench.

7A.11 Braking physics — what “hard brake” means, and why the brake FET must be very low R_{ds}

Braking the car means shorting the motor. A spinning motor is a generator: shorting its terminals lets its back-EMF drive a current back through the windings, and that current produces the retarding torque. The brake current is $I = V_{bemf} / (R_{motor} + R_{brake} + R_{wiring})$, and the retarding torque is proportional to it. In this module R_{brake} is Q2's channel resistance (R_{ds}): Q2 places that resistance across the load (low-side, the BLACK-to-track node to RED), so a “hard brake” is the low-resistance end — Q2 driven toward its minimum R_{ds} .

The two motor types this module serves sit at very different points on that curve, and the brushless case dominates (≈80% of expected use, ISRA classes):

PRIVATE & CONFIDENTIAL

- Brushed motor ($\approx 1.4 \Omega$ winding): R_{motor} is the dominant term. Any reasonable brake resistance is a small fraction of the loop, so the brake is essentially motor-limited and the hardest brake is $\approx V_{\text{bemf}} / 1.4 \Omega$.
- Brushless motor (31–273 m Ω phase): the brake module shorts the on-car Remora ESC's DC bus, and the motor brakes by pushing its back-EMF through the Remora's body-diode rectifier into that short. The loop is roughly two phases in series (~ 60 – $550 \text{ m}\Omega$), plus the ESC's two body-diode drops, plus Q2's R_{ds} . Crucially, that motor resistance is BELOW the brake FET's own on-resistance — so the brake stops being motor-limited and becomes FET-limited.

Figure 7A.11 (left) makes the consequence explicit. “Brake effectiveness” is how close the achieved brake current gets to a perfect zero-ohm short, $\eta = R_{\text{motor}} / (R_{\text{motor}} + R_{\text{brake}})$, independent of speed. In the brushless band: the IRFP250N at a 5 V gate ($\sim 300 \text{ m}\Omega$) is only 9–48% effective — most of the brake is lost in the FET; at a 10 V gate ($\sim 75 \text{ m}\Omega$), 29–78%; a single-digit-milliohm logic-level FET fully on ($\sim 3 \text{ m}\Omega$) reaches 91–99%. For the brushed 1.4Ω motor the IRFP250N is fine (82–100%). The IRFP250N is thus the right device for the case that is shrinking and the wrong device for the case that dominates.

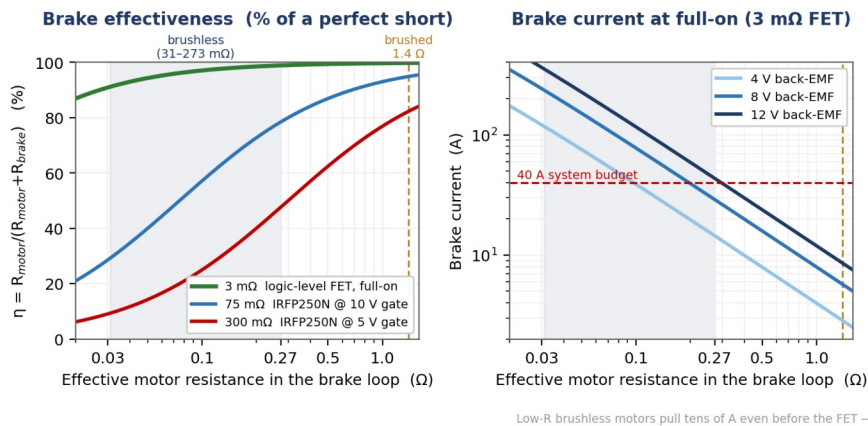


Figure 7A.11 — Left: brake effectiveness (% of a perfect short) vs the effective motor resistance in the brake loop, for three brake-FET options; the shaded band is the brushless phase-resistance range (31–273 m Ω), the dashed line the brushed motor (1.4 Ω). Right: brake current for a 3 m Ω FET fully on, at three back-EMF levels — low-resistance brushless motors pull tens of amps, into the 40 A budget. Effectiveness is speed-independent; the current panel scales with back-EMF.

It is also a thermal necessity, not just an effectiveness one. At these low resistances the brake current is tens of amps (the 40 A system budget is exactly this regime). At 40 A a 3 m Ω FET dissipates $I^2R \approx 4.8 \text{ W}$; the IRFP250N at 75 m Ω would dissipate $\approx 120 \text{ W}$ in the device — it cannot sit in a brushless brake loop at all. The low- R_{ds} FET is forced by both effectiveness and survival.

The brake FET (Q2) should therefore be a single-digit-milliohm logic-level N-channel device — the same class already used in the Remora power stage (TWGMC TW100N03CC $\sim 3 \text{ m}\Omega$, Allpower AP90N03Q, Vishay SiSS-series), driven fully on. For the brushless hard brake this is unambiguous: the FET is fully enhanced (ohmic), its linear-mode SOA is irrelevant, and GD2 on +5 V is sufficient to drive a logic-level gate. The tension is with the BRUSHED linear-resistance tables (§7A.10), which operate the FET in its linear region at a controlled R_{ds} — where modern low- R_{ds} trench FETs have much narrower forward-bias SOA than the planar IRFP250N. Two clean resolutions:

- (a) Validate the chosen low- R_{ds} FET's linear SOA at the brushed operating point (a few amps, a few volts) — the currents are modest, so a suitable part may be fine; or
- (b) Drive the brushed brake by PWM of the same low- R_{ds} FET (always fully on or off, never linear), synthesising the effective resistance by duty. This abandons the original “linear, no switching” choice for the brushed mode, but removes any linear-SOA concern and lets one FET serve both motor types.

Because brushless is 80% of use, the device is chosen low- R_{ds} first; the brushed linear behaviour is the secondary mode to satisfy by (a) or (b).

One brushless-specific caveat: because the brake works through the ESC's body-diode rectifier, there is a low-speed dead-zone — the brake produces no current until the motor's back-EMF exceeds the two series diode drops ($\sim 1.4\text{--}2\text{ V}$). Below that speed the bus short does nothing and the car free-rolls the last little bit. Active phase-short braking in the Remora firmware would avoid this, but that is the ESC's job, not this module's.

The brushless inductance ($2.5\text{--}11.7\text{ }\mu\text{H}$ per phase, $\sim$ doubled line-to-line) sets the brake transient: L/R at these low resistances is tens of microseconds, so current rises fast; stored energy $\frac{1}{2}LI^2$ at 40 A is only $\sim 10\text{ mJ}$, but D2 (the catch diode) must now clamp a 40 A inductive kick on brake release, so its rating wants checking alongside the FET.

Decision: Q2 is selected as a single-digit-milliohm logic-level N-channel (Remora class, e.g. TW100N03CC), driven fully on for the dominant brushless hard brake; GD2 stays on +5 V. The legacy brushed linear-resistance tables are satisfied by keeping linear control: §7A.12 validates that a 30 V low-Rds FET is safe in linear at the brushed operating point (low Vds). The IRFP250N is retained only as an adequate option for brushed-only builds.

7A.12 Validating a low-Rds FET in linear mode (brushed brake tables)

Option (a) from §7A.11 keeps linear resistance control for the brushed tables, which requires the low-Rds FET chosen for the brushless full-on role to also survive linear-region operation. It does — because the brushed brake never drives the FET into the part of the SOA where low-Rds devices are vulnerable.

The brushed envelope is set by the motor ($1.4\text{ }\Omega$) and Q2's table Rds ($1\text{--}25\text{ }\Omega$) forming a divider of the back-EMF (up to $\sim 16\text{ V}$ at brake onset): $V_{ds} = V_{bemf} \cdot R_{brake} / (R_{motor} + R_{brake})$, $I_D = V_{bemf} / (R_{motor} + R_{brake})$, $P = V_{ds} \cdot I_D$. Worst-case dissipation is at the impedance match $R_{brake} \approx R_{motor}$: at 16 V back-EMF that is $\sim 8\text{ V}$ across Q2 at $\sim 5.7\text{ A}$, $\sim 46\text{ W}$ — momentary (highest at brake onset, decaying as the car slows), at perhaps 10–20% duty per lap. Crucially the high-Vds points (soft tables) occur at near-zero current; the locus never combines high Vds with high current (Figure 7A.12).

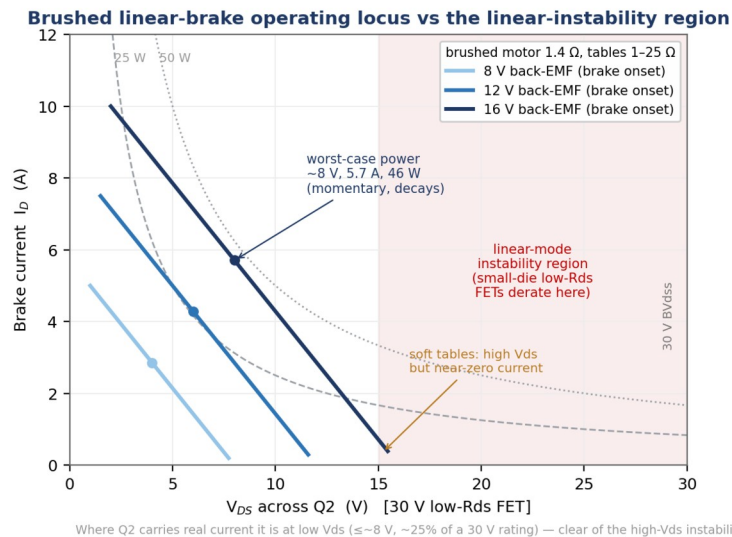


Figure 7A.12 — The brushed linear-brake operating locus (motor $1.4\text{ }\Omega$, tables $1\text{--}25\text{ }\Omega$) for three brake-onset back-EMFs, on a 30 V low-Rds FET. Where Q2 carries real current it sits at low Vds ($\leq 8\text{ V}$, $\sim 25\%$ of the rating); the high-Vds soft-table points are at $\sim 0\text{ A}$. The locus stays clear of the high-Vds linear-mode instability region where small-die low-Rds FETs derate.

Why that is the safe answer: the SOA limit that penalises low-Rds FETs in linear mode is the thermal-instability (“Spirito”) line, not the simple thermal limit. Reaching low Rds needs a small die, so at high Vds the low-gate-overdrive cells show positive thermal feedback (a hot cell conducts more, hogs current, runs away) and the safe boundary drops below the constant-power line. This is exactly the failure mode the wide-SOA planar IRFP250N was chosen to avoid. But the instability is a high-Vds phenomenon — it bites in the upper part of the voltage rating, not at low Vds. On a 30 V part the brushed brake operates at $\leq 8\text{ V}$

where it carries current (~25% of the rating), comfortably below the instability rolloff; the IRFP250N's 200 V wide-SOA margin is for high-Vds linear operation this brake never approaches.

Validation result: a 30 V low-Rds logic-level FET is safe in linear at the brushed operating point — so the brake keeps linear (no-switching) control for brushed and full-on for brushless with one device. The only real constraint is the ordinary thermal limit (Tj from ~46 W peak, intermittent), handled by the existing heatsink.

Two provisos for the final part. Confirm its DC (or ≥ 100 ms) SOA covers $\sim(8\text{ V}, 6\text{ A})$ — trivial for a 90–100 A device — and that its thermal-instability line sits above that point at the heatsink case temperature. The cheap slot-car FETs (TWGMC, Allpower) often do not publish a linear/DC SOA, so prefer a low-Rds part that does — a Vishay TrenchFET Gen V (e.g. SiSS54DN) or an Infineon OptiMOS — to make this a datasheet check rather than a bench gamble; then bench-confirm by holding the worst-case point on the heatsink for the longest brake duration at lap cadence and watching for any runaway. GD2 must also span the gate range that produces the table Rds values, with the safe-default still parking Q2 at brake-on.

7B. Key Design Decisions

Decision	Chosen approach	Alternative considered	Rationale
Q2 safe default behaviour	Pull-up on Q2 gate driver input drives Q2 into braking region when MCU off	No safe default; separate fail-safe circuit	Safe default is the passive R_SAFE/D_SAFE back-EMF brake: speed-proportional, self-limiting, and off when the car is stationary. Essential safety requirement.
Keep-alive suppression	Q1 off when V_BLACK > floor voltage derived from saved_ka_dac and white_mv	Continuous injection regardless of V_BLACK	Only floor below setpoint; no interference with controller above it.
Ramp shape (entry)	Exponential (geometric decay: brake_dac += error $\times$ alpha/256 per ms)	Linear ramp	Fast initial bite then gentle tail. Matches driver expectation on hard braking.
Anti-brake ceiling	2V (DAC_KA_MAX=2480, ~2V at 12–16V rail)	Higher voltage	2V is top of practical range for commercial controllers (0.2–2V typical). Set via Mode B lookup table — pot position determines anti-brake level at all times.
KA floor save timing	Mode C save on toggle-out after 500ms debounce	Immediate pot takeover	500ms debounce prevents accidental saves from toggle contact bounce. No pot handback needed — Mode A pot is always live.
(reserved, was capture button)	No capture button in Rev 11. KA floor saved by toggle gesture (leaving Mode C).	ADC pin (original design, before PA4/PA5 were reassigned to DAC outputs)	Toggle-out is the natural gesture for mode change. No separate button needed. PB3 reserved for future use.

PRIVATE & CONFIDENTIAL

Decision	Chosen approach	Alternative considered	Rationale
Flash storage	Four 64-bit double-words at page 31 (Rev 11). Magic 0xAA270008.	EEPROM emulation	G0 flash requires 64-bit aligned writes. Page 31 reserved by linker. Four DWs store all tuneable parameters.
MCU package	STM32G051K6U6 QFN-32	G031 SOIC-8	QFN-32 provides PA4/PA5 DAC outputs, PB0/PB1/PB3/PB4 GPIO, sufficient for all signals. G051 has 12-bit DAC required for linear mode.
Toggle positions	3-position: A (brake all motors) / B (brushed anti-brake) / C (KA tuning)	2-position or modeless	Mode A is the racing mode for all motor types (brushed and brushless). Mode B adds coupled anti-brake for brushed. Mode C is KA tuning only — set the floor and return to Mode A.
Minimum brake DAC	Minimum brake resistance selectable via SET BRAKE_OHMS, snapping to one of seven non-linearly spaced stored tables (25/15/5/4/3/2/1Ω). No runtime table math.	Hardcoded minimum	Different motors need different minimum braking resistance (e.g. ~3Ω for a 1105 brushless, ~5Ω for Group 12 brushed). Storing seven tables avoids any per-tick table computation while keeping the choice field-selectable.

8. Firmware Build — libopencm3 and Docker Toolchain

8.1 Library choice

libopencm3 is used instead of STM32 HAL. It provides direct register-level access with thin C wrappers, no RTOS, no hidden interrupt overhead, and a deterministic build. The firmware has no dynamic memory allocation and no C++ dependencies.

8.2 G0-specific differences from other STM32 families

- Flash programming requires 64-bit double-word writes (flash_program_double_word). 32-bit writes are not supported on G0.
- DMA requests are routed via DMAMUX1. ADC1 uses DMAMUX request 5.
- DAC1 has two independent channels on PA4 (CH1) and PA5 (CH2). Both enabled simultaneously in the dac_setup() function.

PRIVATE & CONFIDENTIAL

- The project provides stm32g051k6.ld with MEMORY block: FLASH 32K at 0x08000000, RAM 18K at 0x20000000. Page 31 (last 1K of flash) is excluded from the FLASH region and reserved for NV parameter storage.

8.3 Docker toolchain

Build and flash are containerised. The Dockerfile installs arm-none-eabi-gcc and fetches libopencm3 as a git submodule at build time. This ensures a reproducible build environment independent of the host OS.

- `docker compose run --rm brake build` — compiles firmware to `build/aart-brake.elf` and `build/aart-brake.bin`
- `docker compose run --rm brake flash` — flashes via OpenOCD over SWD (ST-Link V2 or compatible)
- `docker compose run --rm brake debug` — starts OpenOCD GDB server on port 3333

8.4 Build workflow

After cloning: `git submodule update --init --recursive` (fetches libopencm3), then `docker compose run --rm brake build`. The build produces `aart-brake.elf` (with debug symbols) and `aart-brake.bin` (raw binary for UART bootloader upload). Binary size is typically 8–12KB of the 32KB flash.

9. Pre-Commissioning Checklist

9.1 Hardware checks before first power-on

Work through these checks before applying track supply voltage for the first time:

- Confirm MCU is STM32G051K6U6 (QFN-32). PA4 and PA5 must route to GD2 and GD1 op-amp buffers respectively.
- Verify WHITE sense divider: R8=18k Ω , R9=4k7 Ω on PA0. At 16V gives 3.31V. If 15k Ω fitted instead of 18k, replace it — 15k gives 3.82V at 16V, exceeding ADC reference.
- Verify BLACK sense divider: R10=22k Ω , R11=4k7 Ω on PA1. Confirm tap is on TRACK SIDE of the pass switch. Essential for correct brake detection.
- Confirm the two N-channel reverse-blocking switch (Q5/Q6 + U5 INA210 + U6 LM311 ideal diode) in the BLACK controller path. Verify the reverse threshold/hysteresis, the gate-drive charge pump, and bench-measure insertion drop at 40A (section 7A.2).
- Confirm D4 (MBRS360 or SS54) in series with Q1 output. D4 is essential whenever Mode A is used with a non-zero KA floor (brushless operation). Without D4, Q1 body diode takes repetitive ~46A current pulses when Q2 hard-brakes while Q1 is trying to maintain the floor.
- Confirm catch diodes D2/D3 are SS310 (100V). SS34 (40V) is not adequate on a 16V rail with inductive motor loads.

- Confirm F1 (polyfuse 500mA–1A) is in WHITE rail supply path only. Not in BLACK track path.
- Confirm J14 (TM1637 display header) pin orientation: pin 1 = CLK (PB4), pin 2 = DIO (PB8), pin 3 = GND, pin 4 = Vcc 3.3V. Confirm J_USB (USB-C) CC1 and CC2 pins each have 5.1kΩ pull-down to GND. Confirm D_USB (BAT54) is oriented with anode at VBUS/F_USB junction and cathode at LDO input. A reversed D_USB will feed track voltage directly to the USB host.

9.2 Scope verification sequence — Mode A (brake, all motors)

S t e p	Condition	Q1 — DAC2 (KA base)	Q2 — DAC1 (brake gate)	Pass condition
1	Power on, V_BLACK = 8V (running)	0V (Q1 off)	0V (Q2 off)	Module transparent
2	V_BLACK drops to 0.3V (brake), pot CCW	0V	~1.5V DC (DAC_BRAKE_SOFT)	Soft brake. Smooth DC, no PWM.
3	Brake active, pot swept CW	0V	Ramps from ~1.5V to ~2.5V DC	Brake resistance decreasing
4	V_BLACK rises to 8V (throttle pickup)	0V	Ramps to 0V (trail release if set)	Release state then PASS
5	Remove MCU power	off (unpowered)	Gate via R_SAFE from drain back-EMF, D_SAFE-clamped (≈0V if stationary)	Safe default brake. Car brakes.

9.3 Scope verification sequence — Mode B (brake + anti-brake, brushed)

S t e p	Condition	Q1 — DAC2 (KA base)	Q2 — DAC1 (brake gate)	Pass condition
1	Running, V_BLACK = 8V	saved_ka_dac (floor) or 3.3V if not set	0V	Running. Q1 applies saved_ka_dac floor if set, else off.
2	V_BLACK drops to 0.3V (brake), pot CCW	0V (Q1 off during brake)	~1.5V DC	Soft brake. Q1 off during braking.
3	Brake active, pot CW	0V	~2.5V (hard	Brake tracks pot. Q1 off.

S t e p	Condition	Q1 — DAC2 (KA base)	Q2 — DAC1 (brake gate)	Pass condition
			brake)	
4	Toggle to Mode C, pot CCW	0V (DAC_KA_OFF — no floor)	Linear per pot	KA tuning. Display shows 0V.
5	Toggle to Mode C, pot CW	~2V (DAC_KA_MAX)	Linear per pot	KA tuning. Display shows ~2V.
6	Toggle to Mode C, tune floor	ka_dac_table[pot_i dx] live	Normal linear brake	Pot dials KA floor. Display shows it.
7	Toggle back to Mode A (500ms → save)	saved_ka_dac applied as Q1 floor	Linear per pot	LED blinks 3×. Mode A racing with KA floor.

9.4 Scope verification sequence — Mode C (KA tuning, brushless)

S t e p	Condition	Q1 — DAC2 (KA base)	Q2 — DAC1 (brake gate)	Pass condition
1	Toggle to Mode C	ka_dac_table[pot_i dx] live	Linear per pot (brake still works)	Display shows live KA floor voltage.
2	Pot CCW in Mode C	DAC_KA_OFF (0V — no floor)	Linear per pot	Display shows 0V. No floor.
3	Pot CW in Mode C	DAC_KA_MAX (~2V floor)	Linear per pot	Display shows ~2V. Q1 live for eCom verification.
4	Toggle to Mode C, tune floor	ka_pending_save armed	500ms debounce	After 500ms: save + LED blinks 3×.
5	Mode A, V_BLACK = 8V	saved_ka_dac floor active	0V	Mode A running. KA floor applied by Q1.
6	V_BLACK drops (brake)	DAC_KA_OFF (Q1 off during braking)	Linear per pot	Braking. Q1 resumes floor after brake exit.

The most critical safety test in all modes is step 5 of the Mode A sequence: remove MCU power and verify Q2 enters its braking region via the pull-up circuit.

9.5 On-track tuning

Parameter	Command	Default	Tuning guidance
Minimum brake	SET RAMP_MS <0	1860	0 = instant hard brake. 50–150ms for wing

PRIVATE & CONFIDENTIAL

Parameter	Command	Default	Tuning guidance
(pot CCW)	–200>		cars. 0–30ms for pan cars.
Anti-brake ceiling	Mode B pot position	DAC_KA_MAX	Coupled with brake via table. CCW = max anti-brake. CW = no anti-brake.
Keep-alive floor	Pot position when leaving Mode C	DAC_KA_OFF	Toggle to Mode C, rotate pot, toggle back to Mode A. Saves after 500ms.
KA floor save timing	SET BRAKE_ENTER <mV>	5000ms	Reduce if brake is slow to engage.
Brake entry threshold	SET BRAKE_ENTER <mV>	500mV	Reduce if brake is slow to engage.
Brake exit threshold	SET BRAKE_EXIT <mV>	1500mV	Raise if brake persists too long.
Brake exit threshold	Set minimum brake resistance in ohms; snaps to nearest stored table (2, 3, 4, 6 or 8Ω). Default 3Ω. Live immediately; SAVE to persist.	1500mV	Raise if brake persists too long.
Ramp-in time	SET RAMP_MS <0 –200>	0ms	0 = instant. 50–150ms for wing cars. 0–30ms for pan cars.
Trail release	SET RELEASE_MS_A/B/C <0–500>	0ms	0 = instant snap-off. 50–200ms for trail braking.
Latvian brake	SET LATVIAN_DVDT <mV/ms>	0 (off)	Enable with 1000. Watch telemetry for false triggers.

10. Three-Mode Design — Rationale and Behaviour

10.1 The braking problem

A slot car motor is a high-inductance load. When the throttle is cut, the motor acts as a generator and back-EMF drives current through the braid and track. An uncontrolled circuit allows the car to freewheel. A resistive brake shorts the motor through a resistor, converting kinetic energy to heat and providing a controllable deceleration force.

The module improves on a simple fixed resistor by providing: variable resistance (pot-controlled), a programmable minimum resistance floor, optional positive anti-brake for corner entry, and eCom keep-alive for brushless motors.

PRIVATE & CONFIDENTIAL

10.2 Mode A — Brake only, all motors

Pot maps linearly to Q2 brake: CCW = DAC_BRAKE_SOFT (~8Ω), CW = DAC_BRAKE_HARD (dead short). No lookup table. Q1 applies saved_ka_dac as a voltage floor whenever V_BLACK drops below it. Default saved_ka_dac = DAC_KA_OFF so Q1 stays off unless a floor has been set via Mode C. This makes Mode A suitable for both brushed motors (no floor) and brushless motors (with floor). Primary racing mode for all motor types.

10.3 Mode B — Brake and anti-brake (brushed)

Pot indexes brake_tables[][] and ka_dac_table[] simultaneously. During braking, Q2 brakes at brake_tables[active][pot_idx] and Q1 stays off. After brake exit, Q1 injects anti-brake at ka_dac_table[pot_idx]. Both values are live from the same pot position via the coupled lookup table. CCW = minimum brake + maximum anti-brake (~2V). CW = maximum brake + no anti-brake. Brushed motors only.

No separate saved value. The anti-brake level is always the ka_dac_table value at the current pot index. The driver adjusts the pot to set the balance between brake and anti-brake. No capture gesture required.

10.4 Mode C — Keep-alive tuning (brushless, setup mode)

Mode C is a setup mode, not a racing mode. You enter it to set the KA floor voltage, then return to Mode A for racing. While in Mode C the display shows the live ka_dac_table value at the current pot position. Normal braking functions during Mode C (pot maps linearly to brake).

When Mode A is used with a non-zero saved_ka_dac, Q1 clamps BLACK_to_track above the floor whenever V_BLACK drops below it. The eCom sees a minimum voltage rather than a brownout. Mode C exists only to set this floor value conveniently while verifying the eCom response live.

On every toggle move to Mode C, the pot immediately dials the KA floor live (ka_dac_table[pot_idx]), both on Q1 and on the display. The driver rotates the pot until the eCom powers up reliably, then toggles back to Mode A. After 500ms the floor is saved. Normal braking via linear pot mapping continues during Mode C.

10.5 Keep-alive floor tuning procedure

- 1. Toggle to Mode C. The display shows the live ka_dac_table value at the current pot position.

- 2. Connect the car to the track with the PSU running. Rotate the pot and watch the display. CCW = no floor, CW = ~2V floor.
- 3. Rotate the pot CW slowly. Watch the display increase. Listen or watch for the eCom to power up (motor twitches, LED on eCom illuminates).
- 4. Continue CW until the eCom powers up reliably, then back off slightly CCW to find the minimum effective floor.
- 5. Toggle back to Mode A. After 500ms the floor is saved. LED blinks 3×. The display returns to track voltage.
- 6. Verify Mode A braking with KA floor active. The car is ready for brushless racing.

11. FET Behaviour Reference

11.0 Linear mode operation — why and how

The brake FET Q2 is operated in its linear (triode) region for the brushed resistance tables — as a voltage-controlled resistor rather than a switch — which is central to the brushed brake and deserves a full explanation. Two qualifications, both new since the brushless-dominant redesign: for brushless motors Q2 is instead driven fully on (minimum R_{ds} , not a controlled mid-point), and the anti-brake pass device Q1 is not a linear-mode MOSFET at all but a TIP147 BJT inside the GD1 constant-current source (§11.1). The linear-mode discussion below therefore applies to Q2 braking a brushed motor.

11.0.1 The three regions of a MOSFET

Region	Condition	Behaviour	Used for
Cut-off	$V_{gs} < V_{th}$	No drain current. FET is an open circuit.	FET off — passthrough mode
Saturation	$V_{gs} > V_{th}$, $V_{ds} > V_{gs} - V_{th}$	Drain current controlled by V_{gs} only. FET acts as a current source.	Switching (digital)
Linear (triode)	$V_{gs} > V_{th}$, $V_{ds} < V_{gs} - V_{th}$	Drain current controlled by both V_{gs} and V_{ds} . FET acts as a voltage-controlled resistor.	Variable resistance — this design

In the linear region: $R_{ds(on)} \propto 1 / (V_{gs} - V_{th})^2$. For a logic-level Q2, $V_{th} \approx 1\text{--}2\text{V}$ and the useful brushed-table control window spans a few hundred mV just above threshold; the 12-bit DAC places hundreds of steps across it at sub-mV resolution — finer than any mechanical pot. (For brushless, Q2 is taken to full enhancement instead of being held part-way.)

11.0.2 Why linear mode, not PWM switching

- Brushless eCom compatibility. 20kHz switching on the braid creates ripple that confuses the eCom ADC and can cause misfires or resets. Linear FET presents smooth, continuous resistance.
- EMC regulations. High-frequency switching requires CISPR 32 / EN 55032 compliance. A slot car track is a long antenna. Linear control has no switching frequency.
- Resolution. PWM effective resistance depends on motor inductance ($L/\text{duty} \times T$), which varies with motor and speed. DAC-driven linear control is independent of motor parameters.

11.0.3 The gate drive challenge in linear mode

- Stability. FET in linear mode has negative thermal coefficient — self-stabilising. Op-amp buffer must be stable with capacitive gate load. Gate resistors R3/R4 (10Ω) isolate op-amp from gate capacitance.
- Safe default. Pull-up must land in the linear region (V_{gs} between V_{th} and full enhancement), not just force gate high. For a logic-level Q2 ($V_{th} \approx 1\text{--}2\text{V}$), the pull-up targets a point just above threshold — light-to-medium braking, not a violent dead-short.

The safe-default pull-up for Q2 intentionally targets the lower end of the linear region. The car brakes but not violently. For competition use, a harder default can be set by adjusting the pull-up divider.

11.1 Q1 — Anti-brake pass device (constant-current source)

Q1 is the pass device of a high-side constant-current source (section 7A.7), not a voltage follower and not a high-side switch. A 0.1Ω sense resistor in the WHITE-rail leg and op-amp GD1 form the loop: the op-amp drives Q1 so the current is held at $I = V_{\text{demand}} / 0.1\Omega$, independent of motor impedance.

- Demand = 0V (DAC2 code 0) → 0A → anti-brake off.
- Demand = 0.3V (DAC2 full) → 3A → maximum anti-brake.
- In between → proportional current. The demand comes from DAC2 (clean DC), divided to 0.30V, and translated to the WHITE rail by GD3.

Q1 is a TIP147 PNP Darlington (TO-220 + heatsink); an N-channel MOSFET (SUP70060E / RFP50N06 / FDP61N20 / SUP50020EL) is the documented alternative. The collector/drain injects through D4 (MBRS360) into the motor node. Because it is a current source, a stalled motor ($\approx$ short on the start line) draws only the set current ($\sim 3\text{A}$), not the unbounded current a voltage source would deliver — the key reason for this topology.

Firmware: the anti-brake setpoint is a DAC2 code (0 = off ... full = 3A), written to DAC_DHR12R2. The pot index maps to the DAC code. Polarity is 0 = off. Worst-case dissipation is $\sim 47\text{W}$ at stall, transient — heatsinked per section 7A.8 using the pulse-energy / thermal-capacitance method.

11.2 Q2 — Brake FET (N-channel, low-side)

Q2 connects BLACK_to_track to RED/GND, gate driven by DAC1 via op-amp buffer GD2. Because $\sim 80\%$ of use is brushless (motor 31–273 mΩ, braked through the Remora ESC's

body-diode rectifier), Q2 is a single-digit-milliohm logic-level N-channel device — Remora class, e.g. TWGMC TW100N03CC (~3 mΩ) — driven fully on for the brushless hard brake. The IRFP250N is retained only for brushed-only builds. See §7A.11 for the effectiveness/thermal analysis and §7A.12 for the linear-SOA validation of the low-R_{ds} FET in the brushed tables.

DAC1 output		Vgate	Q2 state	Rds approx	Motor sees
0 (DAC_BRAKE_OF F)		0V	Off (V _{gs} < V _{th})	Open circuit	No brake
1860 (DAC_BRAKE_SO FT)		~1.5V	Linear region onset	Several Ω	Soft brake (selectable 25/15/5/4/3/2/1Ω)
Intermediate		~2.0V	Mid linear	~0.5Ω	Medium brake
3095 (DAC_BRAKE_HA RD)		~2.5V	Full-on (brushed) / fully enhanced	~5mΩ	Near dead short
PA0	ADC_IN0	WHITE sense		R8/R9 divider. 3.31V max at 16V rail.	
PA1	ADC_IN1	BLACK_to_track sense		R10/R11 divider. Track side of the pass switch.	
PA3	ADC_IN3	Pot wiper		10k linear pot, 0–3.3V. No centre detent.	
PA4	DAC_CH1	Q2 brake gate → GD2		0–3.3V. DAC_BRAKE_OFF to DAC_BRAKE_HARD.	
PA5	DAC_OUT2 (analog)	Anti-brake DAC2 → divider → GD3 → GD1		DAC2 DC, code 0=off to max.	
PA9	USART1_TX AF1	UART TX		J11. 115200 8N1.	
PA10	USART1_RX AF1	UART RX		J11.	
PA13	SWDIO AF0	SWD debug		J10.	
PA14	SWDCLK AF0	SWD clock		J10.	
PB0	GPIO IN	Toggle bit 0		Active LOW, internal pull-up.	
PB1	GPIO IN	Toggle bit 1		Active LOW, internal pull-up.	
PB3	GPIO IN	(reserved)		Previously capture button. Unused in Rev 11. Reserved for future use.	
PB4	GPIO OD out	TM1637 CLK		Display clock → J14 (TM1637)	
PB6	GPIO OUT	Status LED		Blinks 3× on flash save.	
PB8	GPIO OD out	TM1637 DIO		Display data → J14 (TM1637).	

12.3 Toggle switch decoding

PB0 (bit 0)	PB1 (bit 1)	Mode	Behaviour
HIGH (released)	HIGH (released)	Mode A	Brake only. Pot → brake_tables[]. Q1 off.
LOW (pressed)	HIGH (released)	Mode B	Brake + anti-brake. Pot → brake_tables[] and ka_dac_table[] simultaneously.
HIGH (released)	LOW (pressed)	Mode C	Brake + KA floor. On toggle entry, pot index read once → saved_ka_dac.
LOW (pressed)	LOW (pressed)	Invalid	Defaults to Mode A (safe).

13.1 Industry context

In slot car racing, 'brake' refers to the resistance applied across the motor when the trigger is released. 'Ramp-in' refers to how quickly that brake resistance builds. Hard brake (instant full resistance) gives maximum deceleration but can cause the car to snap sideways if the driver is not precise. Soft brake (gradual ramp) is more forgiving and faster for most drivers on most tracks.

13.2 Why positive anti-brake helps

After the brakes release and the driver begins to pick up throttle, there is a brief period where the car's forward momentum has not yet been converted to acceleration. A small positive voltage injection from Q1 provides a forward torque impulse during this transition, helping the car stay planted and balanced on corner exit. Commercial controllers describe this as anti-brake or positive anti-brake, typically in the range 0.2–2V.

13.3 Brake ramp-in

13.3.1 Exponential ramp shape

The ramp-in is exponential rather than linear. Each 1ms tick: $\text{brake_dac} += (\text{target} - \text{current}) \times \alpha / 256$. Alpha = 256 for instant (no ramp), $\alpha \approx 3$ for maximum ramp (200ms). This gives fast initial bite followed by a gentle tail, which matches what drivers feel as natural.

13.3.2 Setting the ramp time (UART only)

Ramp time is a UART-only parameter. The pot is not used to set ramp time — it controls brake and anti-brake levels directly. Use UART:

- SET RAMP_MS <0–200> — 0 = instant hard brake (default). 50–150ms suits most wing cars. 0–30ms suits pan cars.

- SAVE — persists to flash.
- GET — shows current value.

13.3.3 Interaction with positive anti-brake

The ramp-in applies on brake entry. Anti-brake (Mode B) applies on brake exit. They operate at different times and do not interact. A long ramp-in (gradual brake buildup) combined with a moderate anti-brake (gentle exit impulse) gives the most forgiving driving characteristic for club-level drivers on brushed motors.

13.4 Flash storage layout

Double-word	Offset	Contents
DW0	0x00	magic (32-bit) ka_dac (16-bit) ramp_ms (16-bit)
DW1	0x08	0xFFFF brake_enter_mv (16-bit) brake_exit_mv (16-bit)
DW2	0x10	0xFFFF brake_ohms_idx (16-bit) latvian_dvdt (16-bit)
DW3	0x18	rel_ms_c (16-bit) rel_ms_b (16-bit) 0xFFFF rel_ms_a (16-bit)

Magic = 0xAA270008 (Rev 11). Four double-words (DW0–DW3) store ka_dac, ramp_ms, brake thresholds, brake_ohms_idx, latvian_dvdt, and the three release times. Old records with a different magic are silently discarded and firmware defaults apply until SAVE is issued.

13B. Trail Brake Release

13B.1 Concept

Trail braking is the technique of maintaining partial brake pressure while beginning to apply throttle at corner entry. The brakes fade out smoothly as the car transitions from braking to acceleration. The module implements this as a configurable linear release ramp on brake exit.

13B.2 Entry vs release — different shapes by design

Brake entry is exponential (fast initial bite, decaying tail). Brake release is linear (constant fade rate). The linear release is deliberate — a predictable, constant fade that a driver can feel and modulate. An exponential release would feel unpredictable.

Phase	Shape	Rate	Why
Brake entry (ramp-in)	Exponential	Fast initial onset, decaying tail	Immediate grip; gentle progression prevents snap.

PRIVATE & CONFIDENTIAL

Phase	Shape	Rate	Why
Brake release (trail)	Linear	Constant rate per millisecond	Predictable fade the driver can trust during corner entry.

13B.3 Configuration

Command	Default	Range	Typical starting points
SET RELEASE_MS_A <ms>	0 (instant)	0–500ms	Pan cars: 50–100ms. Wing cars: 100–200ms.
SET RELEASE_MS_B <ms>	0 (instant)	0–500ms	Same guidance as mode A.
SET RELEASE_MS_C <ms>	0 (instant)	0–500ms	Brushless: 30–80ms. Longer values may cause eCom issues.

13B.4 Full throttle snap override

If V_BLACK rises above BRAKE_EXIT_MV + RELEASE_SNAP_MV (500mV) — indicating hard throttle application — the release ramp is overridden and the brake snaps immediately to off. Trail braking only operates in the light-throttle transition zone.

13B.5 Telemetry

State field shows RELEASE while trail ramp is active, PASS when complete. Monitor via UART telemetry to tune release time.

14B. Latvian Brake — Emergency Stop on Track Call

14B.1 Concept and use case

The primary use case is a car coming out of the slot — loss of braid contact. The eCom or ESC loses its voltage reference and behaves erratically. A secondary use case is rapid braking on a track call (PSU cut). The firmware detects either event via WHITE rail dV/dt monitoring.

14B.2 Trigger — WHITE rail dV/dt

The WHITE rail voltage is sampled each 1ms tick. If the voltage drops faster than the latvian_dvdt_mv_per_ms threshold (in mV per ms), the Latvian brake fires immediately. This

PRIVATE & CONFIDENTIAL

detects PSU cut (rapid rail collapse) and car-out-of-slot (controller no longer loaded, rail rises briefly then collapses).

14B.3 Behaviour when active

On Latvian brake trigger: DAC_BRAKE_HARD applied immediately. Q1 turned off. State = STATE_LATVIAN_BRAKE. All normal mode logic, ramp-in, anti-brake, and KA floor are bypassed. The hardware pull-up on GD2 holds Q2 in braking region even after MCU browns out.

14B.4 Configuration via UART

- SET LATVIAN_DVDT <mV/ms> — enable with a value such as 1000. 0 = disabled (default).
- Too sensitive: false triggers during normal braking. Increase threshold.
- Too insensitive: misses out-of-slot events. Decrease threshold.
- Recommended starting point: SET LATVIAN_DVDT 1000, then observe telemetry during normal braking to verify no false triggers.

14B.5 Telemetry

State field shows LATVIAN while active. Module exits LATVIAN state when V_WHITE recovers above RAIL_UNDERVOLTAGE_MV (5000mV).

14. UART Debug Interface and Field Flashing

14.1 Overview

USART1 on PA9 (TX) and PA10 (RX) provides a 115200 8N1 serial interface via J11. Two functions: CSV telemetry output at ~10Hz, and a runtime command interface. No bootloader required for parameter changes — all SET commands take effect immediately and SAVE persists to flash.

14.2 SWD debug header (J10)

4-pin 2.54mm header: pin 1 = SWDIO, pin 2 = SWDCLK, pin 3 = GND, pin 4 = +3.3V. Compatible with ST-Link V2, CMSIS-DAP, and J-Link. Used for initial flashing and GDB debugging.

14.3 UART telemetry header (J11)

3-pin 2.54mm header: pin 1 = TX (PA9), pin 2 = RX (PA10), pin 3 = GND. Connect to any USB-UART adapter. Telemetry format: CSV with header line beginning #. State names: PASS, RAMP, BRAKE, RELEASE, AB (anti-brake), KA (keep-alive only), KA_TUN (Mode C tuning), LATVIAN, SAFE.

14.4 Runtime commands

Command	Example	Description
GET ID	GET ID	Returns firmware identity string: #ID: AART-BRAKE-v11-STM32G051
GET	GET	Returns all current parameter values.
SET BRAKE_ENTER <mV>	SET BRAKE_ENTER 500	Brake entry threshold (V_BLACK falling below this = brake active).
SET BRAKE_EXIT <mV>	SET BRAKE_EXIT 1500	Brake exit threshold (V_BLACK rising above this = brake released).
SET KA_MAX <dac>	SET KA_MAX 2480	Maximum KA/anti-brake conduction DAC value.
SET LATVIAN_DVDT <mV/ms>	SET LATVIAN_DVDT 1000	Latvian brake dV/dt threshold. 0 = disabled.
SET RAMP_MS <ms>	SET RAMP_MS 80	Brake ramp-in time 0–200ms. UART-only — not settable by pot.
SET RELEASE_MS_A <ms>	SET RELEASE_MS_A 150	Trail release time for Mode A (0–500ms).
SET RELEASE_MS_B <ms>	SET RELEASE_MS_B 100	Trail release time for Mode B (0–500ms).
SET RELEASE_MS_C <ms>	SET RELEASE_MS_C 50	Trail release time for Mode C (0–500ms).
SAVE	SAVE	Saves all current parameters to flash. LED blinks 3×.
HELP	HELP	Prints command list.

14.5 BOOT0 and NRST buttons

J12 (BOOT0 button → PA14-BOOT0) and J13 (NRST) are 2-pin 2.54mm headers with momentary pushbuttons. To enter the STM32 ROM UART bootloader: hold BOOT0, press and

release NRST, release BOOT0. The device will enumerate on USART1 at 115200 8N1 and accept standard STM32 bootloader protocol frames.

14.6 ROM UART bootloader procedure

- 1. Connect a USB-C cable to the host PC (preferred — uses the onboard CH340N bridge), or a USB-UART adapter to the J11 header (TX, RX, GND) as an alternative. See section 7A.9.6 for the USB-C path.
- 2. Hold the BOOT0 button (J12). Press and release NRST (J13). Release BOOT0.
- 3. Use stm32flash or STM32CubeProgrammer to upload aart-brake.bin.
- 4. Press NRST to restart the device. The firmware runs immediately.

15. Potentiometer Resolution and Control Precision

15.1 The signal chain

Pot wiper (0–3.3V) → PA3 ADC (12-bit, 4096 steps) → firmware maps ADC value to DAC value (12-bit, 4096 steps) → DAC output → op-amp buffer → FET gate voltage → $R_{ds(on)}$ → effective brake resistance.

15.2 Effective resistance — how linear braking works

The FET $R_{ds(on)}$ varies as approximately $1/(V_{gs}-V_{th})^2$ in the linear region. The useful Q2 control window is ~600mV (1.5V to 2.5V gate). This maps to a wide range of resistance from ~8 Ω (soft) to ~5m Ω (dead short). The relationship is highly non-linear — most of the useful variation is in the lower half of the DAC range.

15.3 Resistance resolution across the brake range

At mid-range (DAC $\approx$ 2400): $dR_{ds}/dDAC \approx 0.05\Omega/\text{step}$. At soft end (DAC $\approx$ 1900): $dR_{ds}/dDAC \approx 0.2\Omega/\text{step}$. At hard end (DAC $\approx$ 3000): $dR_{ds}/dDAC \approx 0.001\Omega/\text{step}$. The pot full range (4096 ADC steps) maps to the full 750 DAC steps in the control window — approximately 5:1 mechanical advantage, giving finer effective resolution than the pot travel alone.

15.4 Anti-brake voltage resolution

The Q1 anti-brake control window is also ~600mV (2.0V to 3.3V gate, inverted). 750 DAC steps across this window. Pot full travel CCW to CW maps 0V to ~2V of anti-brake injection voltage. Resolution: 2V / 750 steps $\approx$ 2.7mV per pot step — well below any perceptible threshold.

15.5 The binding constraint: DAC quantisation

With a 12-bit DAC and 750 useful steps in the brake control window, the limiting factor is DAC quantisation (0.8mV/step), not pot mechanical resolution. A standard 10k Ω linear pot has typically 1–10 Ω end-to-end resistance variation and $\pm 1\%$ linearity — far coarser than 0.8mV DAC steps. No centre detent is needed or useful.

15.6 Pot specification recommendations

- Linear taper (not audio/log). The linear taper gives uniform resistance change per degree of rotation.
- 10k Ω end-to-end resistance. Matches ADC input impedance; minimal loading on 3.3V reference.
- No centre detent. A plain linear 10k Ω pot. CCW = minimum brake (and maximum anti-brake in Mode B). CW = maximum brake (no anti-brake). No mode boundary at centre.
- Dust-sealed (IP54 or better) recommended for module enclosure use. The module will be handled at the track.
- Shaft diameter and knob style: to taste. Standard 6mm D-shaft fits most available knobs.

15B. Comparison with Richard Lang's Approach and Table Resolution

Industry note: Roger Schmitt's newest controller uses a single pot for both brake and anti-brake. When the selector is set to anti-brake, it extends the lower range of the brake pot past brake-off into the anti-brake region. This is a cleaner single-control UX than the AART mode-toggle-plus-coupled-table approach and is worth considering as an alternative to Mode B in a future revision.

15B.1 Richard Lang's implementation

The AART Rev 11 lookup table approach (used in Mode B) is inspired by Richard Lang's earlier Arduino-based slot car controller implementation. Understanding the similarities and differences clarifies the design decisions made in the AART module.

Richard's implementation (210922PeteRCarrayAntiBrake) uses a 256-entry PROGMEM array `adcCurve[]` stored in AVR flash memory to give a ratiometric (exponential) characteristic to the brake, acceleration, and deceleration pots. The 8-bit ADC reading is used directly as an array index:

- brake output = `adcCurve[brakeADC]` — maps 0–255 pot reading to 16–255 output with ~16:1 exponential ratio
- anti-brake is handled by a second, separate pot (`antibrkADC`, the repurposed decel pot) producing an independent floor voltage
- the two outputs are combined in `minmaxfunc()`: when the trigger is in the brake zone, instead of returning 0V, the function returns `antibrklimit = min(minimumlimit, antibrkADC)`

Aspect	Richard Lang (Arduino)	AART Rev 11 (STM32G051)
Microcontroller	ATmega32U4 (8-bit AVR, 8MHz RC osc)	STM32G051K6U6 (32-bit Cortex-M0+, up to 64MHz)
Table storage	PROGMEM flash (pgm_read_byte_near())	RAM or flash constant array, no special access required
Table entries	256 × 1 byte = 256 bytes per table	256 × 2 bytes = 512 bytes per table (uint16_t for 12-bit DAC)
ADC resolution	8-bit (256 steps)	12-bit ADC, right-shifted to 8-bit index (256 steps)
Brake output	8-bit PWM (0–255)	12-bit DAC (0–4095; brake spans the selected soft endpoint to 3095)
Anti-brake input	Separate pot (antibrkADC)	Same pot as brake — inverse relationship in table
Anti-brake output	PWM floor set by antibrklimit	DAC floor set by ka_dac_table[idx]
Pot count	2 pots (brake + anti-brake)	1 pot (both outputs from single index)
Capture gesture	None — anti-brake follows pot live	None — both outputs follow pot live (KA floor captured by toggle)
Table shape	Exponential ratio ~16:1 (values 16–255)	Exponential (gamma=1.8); brake: soft endpoint→3095; KA = DAC2 code 0→max (current source)

15B.2 Key design differences

The most significant difference is the relationship between brake and anti-brake:

- Richard’s design: brake and anti-brake are independent. The driver can set hard braking and high anti-brake simultaneously if desired. The two pots allow the characteristics to be tuned independently.
- AART Rev 11: Mode B couples brake and anti-brake through the table (same pot, both outputs live). Mode A is linear brake only for all motors, with saved KA floor from Mode C applied by Q1. Mode C is KA tuning only.

This coupling is deliberate. In slot car racing, very hard braking and very high anti-brake are physically contradictory objectives — the anti-brake torque impulse would immediately counteract the deceleration the driver is trying to achieve. Coupling them through the table enforces a physically sensible relationship with a single control.

The tradeoff is reduced flexibility. A driver who wants to experiment with unconventional combinations (e.g. moderate brake with high anti-brake) cannot do so with a single pot. The

UART interface provides RAMP_MS and RELEASE_MS as compensation — the parameters most likely to need individual tuning are accessible without a physical pot dedicated to each.

15B.3 Should the table be expanded beyond 256 entries?

The STM32G051 has a 12-bit ADC (4096 steps) and 32KB flash. The question of whether to use all 4096 ADC steps as table indices rather than right-shifting to 8 bits (256 steps) deserves a rigorous answer.

15B.3.1 What limits effective control resolution?

Source of variation	Approximate magnitude	Equivalent ADC steps (12-bit)	Implication
Pot mechanical resolution (human hand, $\pm 1^\circ$ of 270° travel)	$\pm 0.37\%$ of range	± 15 steps	Hand cannot reliably set finer than ~ 15 ADC steps
Pot tracking resistance tolerance ($\pm 1\%$ typical)	± 41 steps	± 41 steps	Pot linearity limits meaningful resolution to ~ 100 steps
FET $R_{ds(on)}$ thermal variation ($\pm 20\%$ with temperature)	± 247 DAC-equivalent steps	$\gg 100$ steps	Gate voltage must change significantly before R_{ds} changes noticeably
ADC noise + quantisation	$\pm 2-3$ LSB	± 3 steps	Negligible — not the limiting factor
Table step size at soft end (256 entries)	~ 1 DAC count per step	1 step	Already at the floor of perceptibility

The analysis shows that the human hand and pot mechanics limit effective resolution to approximately ± 15 ADC steps at best. With 256 table entries spanning the full 12-bit ADC range, each table step corresponds to 16 ADC counts — comfortably above the noise floor but below the hand/pot resolution limit. This is the correct operating point.

15B.3.2 Cost of a 4096-entry table

If the ADC were used at full 12-bit resolution with a 4096-entry table:

- Table size: 4096×2 bytes (uint16_t) $\times 2$ tables (brake + KA) = 16,384 bytes = 16KB. Exactly half the STM32G051's 32KB flash budget.
- Step size at soft end: ~ 0.07 DAC counts per ADC step — sub-quantisation, producing long runs of identical table values. Entirely imperceptible.
- Build time: the table would need to be pre-computed and stored as a 16KB constant, or computed at runtime (feasible on a 32-bit MCU, ~ 4 ms at 64MHz for two pow() loops).

A 4096-entry table costs 16KB of flash for zero perceptible improvement. The 256-entry table is the correct choice. The 32-bit MCU's real advantage over the 8-bit AVR is not wider tables — it is the 12-bit DAC (vs 8-bit PWM), the ability to compute table values at runtime rather than storing them in PROGMEM, and the availability of sufficient GPIO for the full circuit without pin-sharing compromises.

15B.3.3 What the 32-bit MCU actually enables

The architectural advantages of the STM32G051 over an 8-bit AVR for this application are:

- 12-bit DAC output. Richard's design uses 8-bit PWM (256 levels) filtered to analogue. The G051 generates a true DC voltage from the DAC with 4096 levels, no filtering capacitor, no PWM ripple on the motor terminal.
- Two independent DAC channels on dedicated pins (PA4 and PA5). No pin-sharing, no timer multiplexing.
- Linear mode gate drive. The DAC outputs a stable DC voltage to the op-amp gate driver. An 8-bit MCU driving a gate driver from PWM requires a filter capacitor and creates residual ripple at the switching frequency.
- Runtime table computation. On the G051, `brake_tables[][]` and `ka_dac_table[]` can be computed in `setup()` using `powf()` rather than stored as PROGMEM constants. This costs ~2ms at startup and saves 1KB of flash. (Currently stored as constants for simplicity.)
- Sufficient GPIO without repurposing ADC pins. Richard's design repurposed the decel pot ADC channel for anti-brake because the AVR ran out of pins. The G051 QFN-32 has enough GPIO for dedicated toggle and LED pins without ADC conflicts.

7C. MCU Selection Review — Could We Use a Smaller Device?

7C.1 Requirements recap

Before evaluating alternatives it is useful to state the minimum peripheral requirements for this application:

Peripheral	Minimum requirement	Why
DAC	2 channels, 12-bit resolution	Gate voltage for Q1 and Q2 in linear mode. 8-bit DAC gives only ~46 steps in the 600mV control window — noticeable coarseness in the soft brake region.
ADC	3 channels, 12-bit	WHITE rail sense (PA0), BLACK_to_track sense (PA1), pot wiper (PA3).
GPIO in	3 pins minimum	PB0/PB1 toggle, PB3 reserved.

Peripheral	Minimum requirement	Why
GPIO out	2 pins	PB4 TM1637 CLK, PB8 TM1637 DIO, PB6 LED.
Timer	1 channel PWM	TIM3 free (was voltmeter PWM on PB4); PB4 now TM1637 display clock, PB8 data.
UART	1x USART	PA9/PA10 for debug telemetry and field parameter updates.
SWD	2 pins	PA13/PA14 for programming and debug.
Flash	~12KB firmware + 1KB NV	Rev 11 firmware is approximately 10–12KB compiled.
RAM	~2KB	Control loop, UART buffers, context structure.

The critical constraint is the dual 12-bit DAC. This is not present on all members of the STM32G0 family and is the primary driver of the current MCU choice.

7C.2 Could we use a smaller STM32G0 package?

The STM32G051 is available in five package sizes: TSSOP20 (F suffix), UFQFPN28 (G suffix), LQFP32 / UFQFPN32 (K suffix), LQFP48, and UFQFPN48. The current device is the K6U6 — UFQFPN32 (5×5mm QFN-32). The F6P6 variant in TSSOP20 (6.5×4.4mm) is the same silicon and appears smaller. However:

The TSSOP20 package has a GPIO bonding problem. In that package, multiple port pins share a single physical pad because there are not enough physical pins to expose them all independently. Specifically: PB3, PB4, PB5, and PB6 are bonded to the same pad, and PB0, PB1, PB2, and PA8 share another pad.

This design uses PB0 and PB1 (toggle switch bits), PB4 (TM1637 display clock), and PB6 (status LED) — all four from the bonded group. They cannot all be used simultaneously from a single physical pin. The TSSOP20 is therefore not a viable package for this circuit without a complete pin reassignment that may not be achievable within the 20-pin constraints.

The UFQFPN32 (QFN-32) used in the current design is 5×5mm with 0.5mm pitch. This is actually easier to solder with hot air than TSSOP20 (0.65mm pitch, 20 pins with exposed leads) because the thermal pad anchors the part and the QFN pads are more tolerant of slight misalignment. The package size difference is marginal and the TSSOP20 offers no assembly advantage for this application.

7C.3 Could we use a DAC-less MCU with an external DAC?

The DAC-less members of the STM32G0 family (G030, G031, G041) are available in smaller and cheaper packages including TSSOP20 and SO-8N (8-pin SOIC). Using one of these with an external dual DAC IC such as the MCP4822 (SPI, 12-bit, dual channel) would in principle work. The trade-offs:

Aspect	STM32G051K6U6 (current)	G031 + external MCP4822
MCU cost	~€0.80 (JLCPCB)	~€0.50 (G031 TSSOP20)
DAC cost	€0 (integrated)	~€0.60 (MCP4822 SOT-23-8)
Total BOM delta	—	+€0.30 — no saving
Extra components	None	MCP4822 + 3× decoupling caps + SPI wiring
Extra GPIO consumed	None	3 (SCK, MOSI, CS) — reduces available GPIO
SPI bus contention	None	Must not conflict with any other SPI device
Startup behaviour	DAC outputs valid immediately at power-on	SPI init required before DAC is valid — gate driver pulls up before MCU sets output
Firmware complexity	dac_setup() sets PA4/PA5 directly	SPI driver + MCP4822 register protocol required
Safe default	Pull-up on op-amp input handles power-off	Same, but startup race condition more complex
libopencm3 support	Full G051 DAC support in tree	SPI library available but external DAC needs manual driver

The conclusion is clear: the external DAC approach costs more (not less), uses more GPIO, adds firmware complexity, creates a startup race condition with the gate driver pull-up, and provides no benefit. It exists as a theoretical option only.

7C.4 Is the 32-bit CPU necessary?

The CPU word size (32-bit vs 8-bit) is unrelated to DAC resolution. The DAC peripheral in the G051 is 12-bit regardless of the CPU. The question of whether an 8-bit MCU with an integrated 12-bit DAC could be used is largely academic — no widely available, JLCPCB-stocked 8-bit MCU has a dual 12-bit DAC integrated. The 8-bit AVR family (used in Richard Lang's reference implementation) has no integrated DAC at all; it uses PWM filtered to analogue, which is specifically what the AART design avoids due to switching noise on the eCom braid.

The genuine advantages of the STM32G051 over an 8-bit AVR for this application are:

- Integrated 12-bit dual DAC. True DC voltage output, no filter capacitor, no PWM ripple on the motor terminal. This is the single most important capability difference.

- Two independent DAC channels on dedicated pins (PA4, PA5). No pin-sharing, no timer multiplexing, no interaction between Q1 and Q2 gate drive.
- Stable DC gate drive. The op-amp buffer receives a steady DC input. No residual switching frequency at the gate, no interference with brushless eCom electronics.
- Sufficient GPIO without pin-sharing compromises. Richard Lang's Arduino implementation had to repurpose the decel pot ADC channel for anti-brake because the ATmega32U4 ran out of dedicated pins. The G051 QFN-32 has enough GPIO for all signals with pins to spare.
- ROM UART bootloader. Field firmware updates without ST-Link — just BOOT0 + NRST and a USB-UART adapter. The ATmega32U4 required avrdude and a specific COM port setup.
- libopenm3 toolchain. Deterministic, no RTOS, no hidden ISR overhead, Docker build, reproducible environment.

7C.5 Recommendation

The STM32G051K6U6 QFN-32 is the correct MCU for this application. It is the smallest package of the only STM32G0 variant that has an integrated dual DAC, sufficient GPIO for all required signals without pin bonding constraints, and is available at JLCPCB. No alternative MCU offers a meaningful advantage:

- Smaller STM32G051 packages (TSSOP20) cannot expose the required GPIO pins simultaneously due to bonding.
- DAC-less MCUs with external DAC are more expensive, more complex, and no smaller in overall board area.
- 8-bit MCUs with integrated DAC are not available in this combination at JLCPCB at competitive price points.

The G051K6U6 at ~€0.80 in QFN-32 represents the optimal point in the cost/capability/size trade-off space for this circuit. The 32-bit CPU, 32KB flash, and 18KB RAM are all substantially in excess of what the firmware requires — but the peripheral set (dual 12-bit DAC + ADC + UART + SWD + timer) is precisely what is needed, and there is no smaller or cheaper part that provides all of it in a single package.

16. Motor Calibration and Brake-Scale Parameterisation (Proposed Feature)

16.1 Motivation

Today the minimum brake resistance is set explicitly with SET BRAKE_OHMS, and the brake is characterised as a resistance running from a chosen minimum down to a dead short. Two problems follow. First, the right minimum is motor-dependent — about 25 Ω for a brushed scale motor, 1–2 Ω for a small one — and currently has to be known and typed in. Second, equal resistance steps do not give equal changes in braking, so the useful adjustment is bunched near the dead-short end. A practice-time

calibration that measures the actual track-plus-lead-plus-motor resistance, together with a brake scale expressed in a quantity that is linear in braking effort, addresses both.

16.2 What the hardware already provides

The module already carries the parts a calibration needs: a programmable 0–3 A anti-brake current source (Q1/GD1, demand set by DAC2 on PA5), V_BLACK_to_track and V_WHITE ADC sensing, and the INA210 reverse-block current monitor (its analogue output, gain 200 across Rsns_sw). Forcing the Q5/Q6 pass switch off for the active method needs an MCU override of the LM311. No new analogue stages are required; the feature is firmware plus the two small connections noted in §16.6.

16.3 Measuring track + lead + motor resistance

The quantity of interest is the total series resistance from the module's BLACK_to_track terminal, through the braid, motor lead and motor winding, back to RED — call it R_loop. Together with the inserted brake resistance it sets the brake current. Two methods are available, of which the second suits “calibrate while practising”:

Active / static. Open the pass switch (assert INP so Q5/Q6 are off, isolating the controller), inject a known current from the anti-brake source into BLACK_to_track with the car held so the drive wheels cannot turn, and read V_BLACK; $R_{\text{loop}} = V_{\text{BLACK}} / I$. Sweep two or three currents and take the slope $\Delta V / \Delta I$ to reject brush drop and any fixed offset. Pulse briefly — Q1 drops most of the 12 V rail at the test current, so the same TIP147 SOA limit as the anti-brake stage applies.

Passive / dynamic. With the INA210 output routed to an ADC channel, sample (V_BLACK, current) pairs while the car is driven normally and extract R_loop from $\Delta V / \Delta I$ across throttle transients. Because the winding's electrical time constant is far shorter than the car's mechanical one, the slope at a fast current step is pure resistance — the slowly varying back-EMF cancels in the difference. This needs no special procedure and calibrates over ordinary practice laps.

It is a two-wire measurement from the module's terminals, but the module's own path (Q5/Q6 $\approx$ 0.6 m Ω , Rsns_sw $\approx$ 1 m Ω) is negligible against the ohms of a brushed motor, so no Kelvin nulling is needed.

16.4 Brushless detection and handling

A static measurement of a brushless car reads the Remora ESC input — its bus capacitance and body-diode — not the brake-relevant behaviour, which is Q2 full-on with braking done by the spinning motor through the ESC rectifier. The two are distinguished by the step response: a brushed motor gives an immediate resistive line, a brushless ESC a capacitive ramp with a diode knee. On detecting brushless the routine reports “brushless — brake is full-on short, no minimum-resistance tuning required” and does not attempt to set a scale.

16.5 Characterising the brake: from ohms to brake effectiveness η

Expressed as resistance, the brake is non-linear and motor-dependent. Brake current is $I = V_{\text{emf}} / (R_{\text{brake}} + R_{\text{loop}})$, so braking effort rises as $1 / (R_{\text{brake}} + R_{\text{loop}})$: near a dead short a small change in R_brake produces a large change in braking, while at high R_brake the same change does almost nothing. That is precisely why fine resolution is needed only near the bottom of the range, and why the “right” minimum shifts with the motor — the resistance axis compresses all the useful travel near zero and stretches it where nothing happens.

A better characterisation is the brake effectiveness already defined in §7A.11: $\eta = R_{\text{loop}} / (R_{\text{loop}} + R_{\text{brake}})$, the fraction of a perfect zero-ohm short. Substituting gives $I = (V_{\text{emf}} / R_{\text{loop}}) \cdot \eta = I_{\text{short}} \cdot \eta$ — braking effort is directly proportional to η . So η is linear in braking force, bounded 0–100% (0% = open, 100% = dead short), and motor-normalised, because R_loop sits inside its definition. Equal steps in η are equal steps in braking everywhere in the range — the uniform granularity the resistance scale cannot give.

The firmware inverts η to the resistance it must insert: $R_{\text{brake}} = R_{\text{loop}} \cdot (1 - \eta) / \eta$. The pot then maps — linearly, or through any deliberately chosen taper — onto η from a gentle minimum up to 100% (dead

short), and the calibration's measured R_{loop} fixes the resistance for each η . As an illustration, with $R_{\text{loop}} \approx 1.4 \Omega$ the present 25/15/5/4/3/2/1 Ω ladder corresponds to $\eta \approx 5 / 9 / 22 / 26 / 32 / 41 / 58\%$, then a jump straight to 100% at dead short: the existing table is densest where the change is barely felt and leaves a 42-point gap across the most sensitive region. A uniform η ladder (for example 10% steps to 100%) puts the resolution where it is actually used — and needs sub-ohm resistances there that are awkward to name in ohms but trivial in η .

Recommendation. Characterise the inserted quantity to the user as brake effectiveness η , a normalised 0–100% braking-effort scale, while keeping ohms as the internal, physical description of what Q2 inserts. The spec can still say honestly “a resistance from a minimum down to a dead short”; the control and the displayed setting are η . η is the natural reparameterisation of the existing open-loop resistive brake — it needs no hardware change, fixes the granularity and the motor-dependence, and consumes the calibration result directly.

16.6 Hardware deltas

Two small additions enable the above: route the INA210 output (reverse-block current) to a spare MCU ADC channel for the passive method and any current readback, and provide an MCU GPIO that can force the pass switch open (override the LM311) for the active method. Both are inexpensive on the QFN-32 and are not on the current netlist; add them if the feature is adopted.

16.7 Future work: closed-loop current-mode braking

η linearises the setting, but the brake is still open-loop — the actual inserted resistance is Q2's linear R_{ds} , which is soft and temperature-dependent (§7A.11). With the INA210 output already providing the main-path current, the more robust end state is to regulate brake current (or deceleration) to a setpoint rather than set a resistance at all: constant-current braking gives constant retarding torque independent of speed and of Q2's characteristics, and the calibration's job becomes choosing a target current, with R_{loop} bounding the limits. This is a larger change — a current control loop to design for stability and bandwidth — but the measurement hardware and the η work are the natural first steps toward it.

16.8 Safety and limits

Active injection makes the motor want to turn: require the car held or the drive wheels off the rail, keep test pulses brief and within the TIP147 SOA, and abort if the measured voltage shows a back-EMF offset (rotor spinning). The passive method avoids this entirely. In all cases the brake FET stays off during measurement, so the injected or measured current flows only through the motor loop.

17. Feature additions and roadmap (Rev 11.x)

This section records the firmware features added in this revision and a short roadmap of capabilities evaluated alongside them. Three suggestions — internal-MCU-temperature thermal derating, a hardware-comparator Latvian trigger, and a brake-chatter emulator — were evaluated and rejected as resting on incorrect premises (the MCU die sensor does not see the op-amp or pass-device temperatures; a comparator detects a level, not dV/dt , and the linear gate chain cannot switch in nanoseconds) or as fighting the linear-control architecture; they are not pursued.

17.1 Implemented this revision (firmware)

D — Brake ramp curves. SET RAMP_CURVE 0|1|2 selects the brake-in shape: 0 = linear (constant step), 1 = exponential (fast initial bite — the previous fixed behaviour, still the default), 2 = logarithmic (slow start, late bite). The pot→DAC map already applies a gamma curve; this shapes the time-ramp instead. The choice persists to flash.

E — Binary scope telemetry. SCOPE 1 switches USART1 from the 100 ms CSV stream to a compact ~200 Hz little-endian binary frame (sync 0xA5 0x5A, then tick, state, mode, white_mv, black_mv, brake_dac, ka_dac, pot_raw and a flags byte); SCOPE 0 reverts. It reuses the running DMA-fed ADC and is intended for bench work and advanced users plotting brake entry/exit, Latvian triggers and anti-brake timing.

G — Dual status LED. A green LED on PB5 is added beside the existing red LED on PB6: green steady = drive/pass-through, red = brake or fault, slow green blink = keep-alive floor active; the save-acknowledge blink remains on red. Hardware delta: one LED plus a ~330 Ω resistor on PB5. Two things must be confirmed before layout. First, that PB5 is free on the final QFN-32 schematic — the QFN-32 was chosen specifically to avoid the TSSOP20 pin-bonding problem, so PB3–PB7 are independent here, which is favourable. Second, the BOOT0 wiring: this document and the firmware comments refer to a “PB8/BOOT0” button, but on STM32G0 BOOT0 is the PA14-BOOT0 pin, not PB8. These references are corrected in this revision — the firmware comments and this document now read PA14-BOOT0 (with option bit nBOOT_SEL=0) — and the schematic’s BOOT0 note is fixed; the net rewire (J12 → PA14 through a series resistor) remains a KiCad TODO, after which PB8 is a spare GPIO.

J — Pot auto-zero. An explicit CAL POT_MIN command captures the current (fully-CCW) pot reading as a stored zero offset; the pot is then offset and rescaled to full span, so wear or mechanical bias at the CCW stop cannot produce an unexpected value. It is a deliberate command rather than a silent boot-time capture, so it cannot mis-zero from an unintended pot position. Enabling reliable persistence exposed, and required fixing, a pre-existing flash-record bug: flash_load() read each saved double-word’s high 32-bit half where flash_save_all() had written the data in the low half, so only ka_dac and ramp_ms actually survived a power cycle while brake_enter/exit, brake_ohms, latvian_dvdt and the release times silently reverted to defaults. flash_load() now mirrors flash_save_all() exactly, and pot_min and ramp_curve occupy the previously-unused high word of DW1. This change needs bench validation against a unit with existing saved flash.

17.2 Proposed / documented (not yet implemented)

A — Car profiles. A good idea, kept on the roadmap rather than built for now. Store three complete tuning sets (brake table, KA floor, ramp and curve, thresholds, release times, pot zero) and recall them between heats. It is firmware-only — flash page 31 has room for a multi-profile record — but the value hinges entirely on the recall UI, which is the unsolved part: it must be impossible to trigger by accident mid-race and must not collide with the A/B/C mode semantics, and each profile still has to be set up via Mode C and UART. Candidate mechanisms (a deliberate Mode-C toggle gesture, boot-time selection, or a UART command) each have trade-offs; until one is chosen and felt out on the bench, this stays documented, not implemented.

F — Auxiliary input on PB3. The board already reserves PB3 with the R6/R7 divider DNF. Populating R6/R7 and adding a 2-pin header gives a logic input usable as a lap-counter sensor (reed or IR) or an external “marshal” brake-override; an output to drive a lane-change solenoid would need an external driver and is out of scope for this board. Low effort. The TIM2 timer freed by the DAC2 migration is available for input-capture lap timing if precise timestamps are wanted.

H — Richer UI / data display. The single 4-digit display shows one quantity at a time and is ambiguous when repurposed (volts versus ohms versus amps on an unlabelled field), and overloading the mode toggle for hidden display modes collides with the A/B/C semantics. A better UI — for example a small addressable display, or a defined set of LED codes — is needed to surface more state (brake resistance, anti-brake current, active profile, calibration status) clearly and without that ambiguity. Captured here as a requirement rather than a committed design.

Rev	Date	Author	Changes
26.0	Jun 2026	AART	Rev 11.x firmware features. Implemented: D ramp curves (SET RAMP_CURVE 0 1 2), E binary scope telemetry (SCOPE), G dual status LED (green PB5 beside red PB6), J explicit pot auto-zero (CAL POT_MIN). Corrected a flash_load() double-word offset bug so all SET params persist; pot_min/ramp_curve added to the flash record DW1. Added section 17

Rev	Date	Author	Changes
			(roadmap: A car profiles, F PB3 aux input, H richer UI). Flagged PB8/BOOT0 vs PA14-BOOT0 on G0. Earlier this revision: anti-brake demand migrated to DAC2 (C_KA removed); section 7A.8.1 adds the case-to-sink TIM term ($R\theta_{cs}$) to the thermal calc. Corrected BOOT0: it is PA14-BOOT0 on G0, not PB8 (firmware, doc and schematic note fixed; J12→PA14 net rewire is a KiCad TODO; PB8 freed). Replaced the PB4-PWM analog voltmeter with an MCU-driven TM1637 4-digit display (CLK=PB4, DIO=PB8); TIM3 and the RC proxy are removed.
25.0	Jun 2026	AART	Rev 11 firmware + document. Mode A: brake only all motors (linear pot, saved_ka_dac KA floor applied by Q1 — makes Mode A safe for brushless when floor set). Mode B: brushed anti-brake, lookup table couples brake + anti-brake simultaneously. Mode C: KA tuning only (brushless), pot dials floor live on Q1 and the display, save on leaving after 500ms debounce. D4 essential for brushless Mode A. All stale mode descriptions, SET button references, handback timer references, and capture state references removed from sections 2, 3.6, 9, 10, 12, 13, 14. Figure 2 SVG redrawn for Rev 11.
24.2	Jun 2026	AART	Added section 7C: MCU selection review. Covers peripheral requirements, STM32G051 TSSOP20 GPIO bonding problem (PB3-PB6 bonded, rules out TSSOP20), external DAC option analysis (no cost saving, more complex, startup race), 32-bit vs 8-bit CPU discussion, and recommendation confirming G051K6U6 QFN-32 as optimal choice.
24.1	Jun 2026	AART	Added section 15B: comparison with Richard Lang Arduino implementation (table, two-pot vs single-pot, coupled vs independent outputs), and table resolution analysis (why 256 entries is correct on a 32-bit MCU, 4096-entry table costs 16KB for zero improvement, what the G051 actually enables over AVR).
24.0	Jun 2026	AART	Rev 10 firmware. Radical simplification: single pot indexes into 256-entry lookup tables (brake_tables[], ka_dac_table[]) producing both brake and anti-brake/KA outputs simultaneously. No capture button, no handback timer, no frozen brake, no capture states. KA floor captured by toggle move to MODE_KA (reads pot index once, saves, LED blinks, 2-sec voltmeter display). SET button (PB3) removed. saved_antibrake_dac, saved_min_brake_dac, saved_handback_ms, frozen_brake_dac, capture fields all removed. Flash magic 0xAA270008. Sections 2, 3.6, 4, 9, 10, 12, 13, 14, 15 updated. Lookup table design inspired by a previous implementation by Richard Lang.
22.0	Jun 2026	AART	Major UX redesign: pot controls voltage setpoints in all modes. Mode A pot = brake resistance during braking, capture button + pot = anti-brake ceiling. Mode B pot = brake floor, button captures it. Mode C unchanged. Ramp-in and trail release UART-only. Firmware: handle_capture() split into per-mode handle_capture_a/b/c(). BrakeCtx_t adds saved_antibrake_dac, saved_brake_floor_dac, capture preview fields. Flash magic 0xAA270005, DW4 added. SVG diagram AART_Brake_UI.svg added.
21.0	Jun 2026	AART	Added mini voltmeter interface: section 3.7 with display mode table, circuit description (TIM3_CH1 PB4, R_VM 10k, C_VM 100uF, D_V SS310), J14 3-pin connector pinout, firmware PWM scaling. BOM updated. KiCad updated.
20.1	Jun 2026	AART	Added section 7A.6: Power-down, brownout and hold-up capacitor. STM32G051 POR/PDR/BOR supervisor table. Current budget comparison.

PRIVATE & CONFIDENTIAL

Rev	Date	Author	Changes
			Worked hold-up cap calculation ($C=I \times t / \Delta V$, 100 μ F). Component selection table. Power-down sequence table.
20.0	Jun 2026	AART	Section 11 completely rewritten. New section 11.0: linear mode operation — three MOSFET regions, why linear not PWM, gate drive stability and safe-default challenge. Sections 11.1/11.2/11.3 rewritten with correct DAC values and D4 current path analysis.
19.2	Jun 2026	AART	GD1 ordered before GD2 in component table.
19.1	Jun 2026	AART	Q1 ordered before Q2 in all tables. Consistency audit fixes: section 7B CCR→DAC language, capture button PA5→PB3, flash record description, safe-default mechanism; section 8.2 linker script stm32g041j6→stm32g051k6, RAM 8K→18K; section 9 DAC column headers and data rows reordered.
19.0	Jun 2026	AART	Added section 3.6 User Interface. Removed section 7.0 prototype strategy table. Section 9 completely rewritten for DAC-based verification.
18.0	Jun 2026	AART	Added trail brake release (section 13B): linear release ramp on brake exit, configurable per mode via SET RELEASE_MS_A/B/C (0-500ms). Added GET ID UART command. Flash magic 0xAA270004. STATE_RELEASE added.
17.0	Jun 2026	AART	Added section 1.3 Circuit Topology with Figure 1 (AART_Brake_Topology.svg). Six explanatory bullets. Topology diagram matches hand-drawn schematic.
16.0	Jun 2026	AART	Production design adopted. TIM3 PWM removed, replaced by DAC1 CH1/CH2. Two N-channel reverse-blocking switch (TPS1212-Q1) in BLACK path. R10/R11 BLACK sense on track side of the pass switch. Toggle/capture moved to PB0/PB1/PB3. Firmware Rev 7.
15.3	Jun 2026	AART	Section 1.1 Purpose rewritten: three design drivers (fine control, Group 12 anti-brake, eCom keep-alive floor).
15.3	Jun 2026	AART	Reverse-block driver changed from the TPS1212-Q1 to an INA210 + LM311 ideal diode (~2 μ s, ~0.16A trip, ~50mV hysteresis) to cut the brake-contact current glitch ~13 $\times$ and avoid browning out the eCom. Drops the TPS1212 I ² t/SCP/IMON protection (open item); the gate-drive charge pump, the ~2 μ s speed against real INA210+LM311 models, and the 26V INA210 common-mode ceiling are open items. SKiDL netlist, KiCad sheets, topology and README updated to match; firmware I_DIR/PA6 logic unchanged.
15.2	Jun 2026	AART	Corrected conflation of positive anti-brake (injection, modes A/B) and eCom keep-alive floor (brownout prevention, mode C).
15.1	Jun 2026	AART	STM32G051K6U6 adopted as current MCU. G051 has DAC for linear control, same QFN-32 package. Linker script renamed stm32g051k6.ld.
15.0	Jun 2026	AART	Q1/Q2 and GD1/GD2 renumbered to match topology diagram. Reverse-blocking pass switch added to BLACK path. PA6/PA7→PA4/PA5 DAC. All component tables updated.